

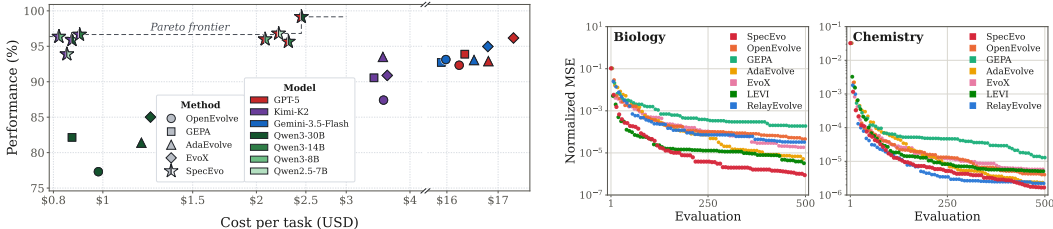
SPECEVO: SPECULATIVE EVOLUTION WITH LARGE LANGUAGE MODELS FOR COST-EFFICIENT SCIENTIFIC DISCOVERY

Anonymous authors

Paper under double-blind review

ABSTRACT

LLM-driven evolutionary search is a promising paradigm for scientific discovery, but existing methods remain costly and slow, often applying expensive reasoning uniformly while underusing evidence from failed, redundant, and saturated trials. Recent cost-aware approaches allocate models through predefined roles or one-way handoffs, but do not continually adapt this coordination as search evidence accumulates. We introduce *Speculative Evolution* (SPECEVO), an LLM-powered framework for efficient and fast scientific discovery that makes model coordination history-dependent and self-correcting. Lightweight Speculators perform parallel exploration, while an Advisor revises persistent guidance from trajectory-wide successes, failures, and saturated directions. At sparse checkpoints, a frontier-model Navigator inspects the evolving population and selects among synthesis, surgical refinement, and reframing. This *speculate-then-consolidate* workflow amortizes costly reasoning across many subsequent evaluations. Across 175 tasks in four scientific discovery domains, SPECEVO matches or outperforms frontier-model baselines while reducing API cost by up to 80%, converges up to $1.5\times$ faster than the cost-efficient baselines on equation discovery, and improves over the strongest CO-Bench baseline by 3.8%. These results demonstrate the effectiveness of trajectory-conditioned model coordination for efficient scientific discovery.



(a) Performance–cost trade-off across methods and backbone configurations on mathematical discovery tasks. (b) Best-score trajectories for equation discovery in the Biology and Chemistry domain.

Figure 1: Existing LLM-driven evolutionary methods remain costly and slow: performance is tightly coupled to backbone strength, while evidence from failed, redundant, and saturated trials is insufficiently consolidated to guide subsequent proposals. SPECEVO mitigates both limitations through low-cost speculation and sparse, evidence-driven consolidation.

1 INTRODUCTION

Recent advances in large language models have given rise to a new paradigm of evolutionary search for automated scientific discovery (Romera-Paredes et al., 2024; Lu et al., 2024b; Yamada et al., 2025; Liu et al., 2026b). In this paradigm, LLMs act as semantic variation operators, proposing meaningful modifications to candidate programs that are iteratively refined through execution-based evaluation and selection (Liu et al., 2026a; Wu et al., 2024). Such approaches have achieved strong

results across mathematical discovery, combinatorial optimization, systems engineering, and program synthesis (Hieu et al., 2026; Cemri et al., 2026; Novikov et al., 2025; Liu et al., 2024b).

Despite their strong results, current LLM-driven evolutionary systems often conflate search quality with backbone capability (Agrawal et al., 2025; Yan et al., 2026; Wang et al., 2025). Many frameworks rely on a single backbone, or a nearly uniform model policy, across routine local edits, invalid-program repair, and major strategic changes. This makes architectural gains difficult to isolate from model strength and requires repeated frontier-model calls, making full search runs costly and difficult to reproduce under realistic API and wall-clock budgets (Liu et al., 2026c) (Figure 1a). Recent work mitigates this dependence through adaptive search policies (Cemri et al., 2026; Liu et al., 2026b), structured strategy-level memory (Luo et al., 2026b), and multi-model routing (Tanveer, 2026; Luo et al., 2026a). However, these advances still make limited use of accumulated search evidence to guide subsequent decisions. Lightweight models may repeatedly revisit similar patterns (Dat et al., 2025) or generate invalid programs (Liu et al., 2024a); when failures, near misses, and saturated directions are discarded or passively stored rather than distilled into actionable guidance, they cannot improve future proposals (Luo et al., 2026b), leading to slow convergence or early plateaus (Figure 1b). Existing routing schemes only partially close this feedback loop: RelayEvolve (Luo et al., 2026a) performs a one-way handoff from cheap exploration to strong-model refinement, whereas LEVI (Tanveer, 2026) invokes the stronger model primarily for predefined paradigm shifts. What remains missing is a closed-loop architecture that continually uses trajectory-wide evidence to update both lightweight exploration and expert intervention.

We address this gap with SPECEVO, an evolutionary architecture inspired by the collaborative workflow of a resource-constrained real-world research laboratory and built around a *speculate-then-consolidate* (STC) workflow. Lightweight Speculators continuously generate and evaluate candidates through complementary variation operators. An adaptive behavioral archive and shared search history jointly retain high-performing programs, rejected proposals, execution failures, and niche-level progress. At slower cadences, the Advisor compresses recurring errors and saturated directions into persistent guidance, while the Navigator reasons over the population as a whole and selects a synthesis, surgical, or reframe intervention according to the search state. STC therefore delays expensive reasoning until sufficient evidence has accumulated for one intervention to influence many subsequent evaluations, rather than spending frontier-model capacity on every mutation. Across model configurations and budget settings, SPECEVO consistently matches or surpasses strong LLM-driven evolutionary baselines while substantially improving search efficiency. Under the same wall-clock budget, it reduces API cost by approximately 50%–80% across mathematics, systems, and combinatorial optimization, while scaling additional API budget yields gains of up to 5%. On symbolic regression, SPECEVO further dominates all baselines under an identical evaluation budget, converging up to $1.5\times$ faster. The main contributions of this work are as follows:

- We introduce SPECEVO, a resource-aware evolutionary architecture that closes the loop between low-cost exploration and sparse expert reasoning through continual consolidation of accumulated search evidence.
- We develop *speculate-then-consolidate* (STC) mechanism, which turns archive- and trajectory-level evidence into two complementary feedback mechanisms: persistent, failure-aware guidance for subsequent speculation and search-state-dependent population-level reasoning that determines how the frontier model should intervene.
- We benchmark SPECEVO on 175 diverse tasks across mathematical, systems, combinatorial optimization, and scientific equation discovery with multiple benchmark–budget settings. SPECEVO consistently matches or outperforms strong baselines across model configurations and resource regimes while substantially reducing API cost.

2 RELATED WORK

LLM-Guided Evolutionary Search. LLMs increasingly serve as semantic variation operators that refine executable artifacts through evaluation and selection. FunSearch established this paradigm for mathematical discovery (Romera-Paredes et al., 2024), while AlphaEvolve broadened it to larger codebases and scientific and systems problems (Novikov et al., 2025). Subsequent frameworks strengthen different parts of the loop: OpenEvolve combines island evolution with quality-diversity archiving (Sharma, 2025); GEPA uses reflection and Pareto selection (Agrawal

et al., 2025); SeaEvo maintains persistent strategy-level memory (Luo et al., 2026b); AdaEvolve adapts exploration and resource allocation under stagnation (Cemri et al., 2026); and EvoX evolves the search policy itself (Liu et al., 2026b). These methods primarily advance candidate generation, representation, or search control. SPECEVO instead focuses on repeatedly consolidating trajectory-wide evidence to coordinate heterogeneous reasoning roles operating at different cadences.

Cost-Efficient LLM Allocation. Cost-efficient LLM systems commonly coordinate cheap and strong models through cascades or adaptive routing (Chen et al., 2023b; Aggarwal et al., 2024; Ding et al., 2024; Ong et al., 2025). SCOPE uses query-level partial evaluations to select a fixed model assignment for each module of a compound system under a quality constraint (Huang et al., 2026). While reducing configuration-search cost, it does not adapt allocation to an evolving search state. Within evolutionary search, LEVI combines diversity preservation and proxy evaluation with role-aware routing, assigning local refinements to cheap models and paradigm shifts to a frontier model (Tanveer, 2026). However, the frontier model remains tied to a predefined role, while its error-oriented advice does not use broader progress and saturation evidence to adapt interventions. RelayEvolve develops multiple cheap-model trajectories and performs a one-time handoff of a quality-diverse population to strong-model refinement (Luo et al., 2026a). Afterward, lightweight exploration ends, and evidence from discarded or exhausted trajectories is not consolidated into reusable guidance. SPECEVO instead continually consolidates accumulated evidence to guide both lightweight exploration and state-dependent frontier-model intervention.

3 METHODOLOGY

Figure 2 illustrates the end-to-end workflow of SPECEVO. See Appendices E, I for algorithmic details and prompt templates.

3.1 MOTIVATION: FROM SPECULATIVE DECODING TO SPECULATIVE EVOLUTION

Speculative decoding exploits an asymmetry in language generation: a lightweight draft model proposes several tokens, while a stronger target model verifies them in parallel, amortizing expensive inference without changing the target distribution (Leviathan et al., 2023; Chen et al., 2023a). Recent evolutionary methods similarly show that lightweight models can handle routine refinements, reserving stronger models for uncertain mutations or structural shifts (Ray et al., 2026; Tanveer, 2026). Scientific search, however, is more than draft-and-verify: it unfolds over a stateful trajectory whose successes, failures, near misses, and saturated directions must inform future proposals.

SPECEVO transfers the resource-allocation principle of speculative decoding from the token level to the trajectory level, organized as a resource-constrained research laboratory. Lightweight **Speculators** act as junior researchers, continuously generating candidates for execution-based evaluation. The **Advisor** acts as a postdoctoral researcher, periodically consolidating trajectory-wide evidence into persistent guidance. The frontier-model **Navigator** acts as the principal investigator, intervening sparsely to redirect the population. We call the resulting workflow *speculate-then-consolidate* (STC): expensive reasoning is deferred until enough evidence has accumulated for a single intervention to influence many subsequent low-cost evaluations.

3.2 THE SPECULATE-THEN-CONSOLIDATE WORKFLOW

SPECEVO interleaves three reasoning processes at different cadences. After initialization, W parallel Speculators (four in our experiments) continuously draw parent programs from the archive, apply one of six variation prompts using a lightweight model $\mathcal{M}_S$, execute the resulting candidates, and record their outcomes, handling most candidate generation through this low-cost loop. Every Δ_N completed evaluations ($\Delta_N=20$), the Navigator ($\mathcal{M}_N$, the frontier model) inspects a compact population view and makes one strategic intervention—recombining leading solutions, repairing the current champion, or introducing a new strategy family—according to the search state. Every Δ_A evaluations ($\Delta_A=50$), the Advisor ($\mathcal{M}_A$) revises a short guidance note from accumulated successes, failures, and saturated directions for injection into a fraction of subsequent Speculator prompts. These processes and periodic archive reclustering maintain independent evaluation clocks (Appendix E). We next define the shared search state and each functional role.

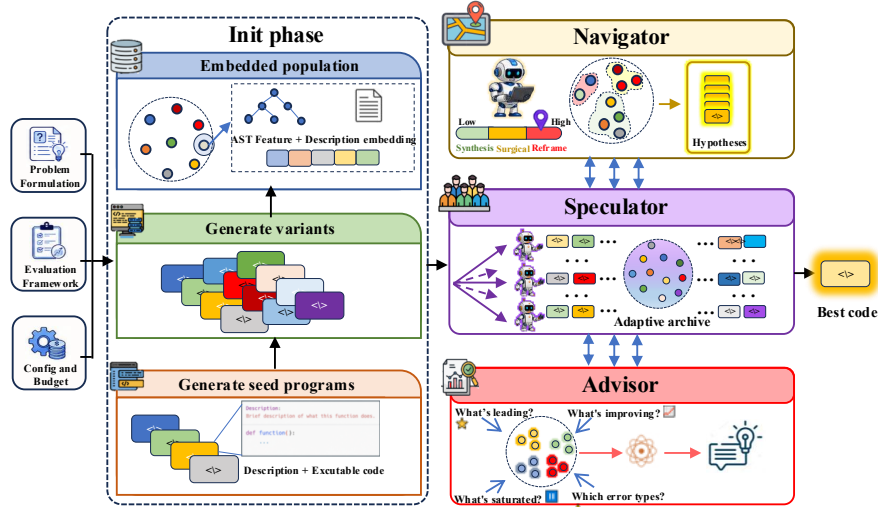


Figure 2: Overall of SPECEVO’s workflow. Following diverse initialization, lightweight Speculators continuously explore an adaptive behavioral archive, while the Advisor distills trajectory-wide experience into reusable guidance and the Navigator sparsely intervenes according to the evolving population state.

Problem setup. A task $\mathcal{T}$ specifies a space $\mathcal{P}$ of executable candidate programs and an evaluator $F : \mathcal{P} \rightarrow \mathbb{R}$. The objective is to discover $p^* = \arg \max_{p \in \mathcal{P}} F(p)$ under a prescribed wall-clock, API-cost, or evaluation budget. At evaluation step t , SPECEVO maintains two shared structures: a behavioral archive $\mathcal{A}_t$ containing the best program discovered in each niche, and a search history $\mathcal{H}_t$ recording every evaluated attempt, including rejected candidates and execution failures. Each stored program p is paired with a short natural-language description $d(p)$ of its strategy, produced by the model that generated it.

Stagnation signal. We summarize recent search progress using a scalar stagnation signal,

$$s_t = \max \left\{ \min \left(1, \frac{t - \tau_t^{\text{best}}}{H_g} \right), \min \left(1, \frac{t - \tau_t^{\text{adm}}}{H_l} \right) \right\}, \quad (1)$$

where τ_t^{best} and τ_t^{adm} denote the most recent global-best improvement and archive admission, respectively, with normalization horizons $H_g=100$ and $H_l=20$. Thus, s_t increases from 0 toward 1 when either the incumbent best or the broader archive ceases to improve. This signal controls both parent sampling and the Navigator’s intervention mode below.

Behavioral archive. $\mathcal{A}_t$ is a MAP-Elites-style archive (Mouret & Clune, 2015) whose niches are induced by K -means clustering ($K=50$) in a data-driven structural–semantic descriptor space. At reclustering round r , each program is represented as

$$\mathbf{z}_r(p) = [\phi(p) \parallel \mathbf{U}_r^\top (\mathbf{e}(p) - \bar{\mathbf{e}}_r)], \quad (2)$$

where $\phi(p)$ contains eight log-scaled AST statistics and $\mathbf{U}_r^\top (\mathbf{e}(p) - \bar{\mathbf{e}}_r)$ projects the description embedding $\mathbf{e}(p)$ of $d(p)$ onto its leading principal directions. The structural and semantic blocks are z-scored separately against the current population so that neither dominates the distance metric. A new candidate is assigned to its nearest centroid and admitted only if the corresponding niche is empty or the candidate strictly improves upon its incumbent. Because the population geometry evolves throughout the search, the PCA projection is re-fitted and the K -means clustering is updated every $\Delta_C=50$ evaluations, with centroids warm-started from the previous clustering (Appendix E).

Initialization. Effective speculation requires a diverse set of verified starting hypotheses. The Navigator generates $M=5$ seed programs sequentially. Each call is conditioned on the strategies and evaluation outcomes of the previously generated seeds and is explicitly prompted to introduce a strategy family not yet represented, rather than producing independent samples that may collapse onto the same approach. The Speculator pool then expands each executable seed into $V=20$ inexpensive variants in parallel, using one or two other seeds as inspiration. The behavioral archive is initialized from the resulting evaluated population.

Speculators: continuous exploration. An idle Speculator selects a parent according to its fitness rank r_i under a Zipf distribution whose exponent adapts to the search state (Piantadosi, 2014):

$$\Pr(p_i | \mathcal{A}_t) = \frac{r_i^{-\beta_t}}{\sum_{j=1}^{|\mathcal{A}_t|} r_j^{-\beta_t}}, \quad \beta_t = \beta_{\min} + (1 - s_t)(\beta_{\max} - \beta_{\min}), \quad (3)$$

with $(\beta_{\min}, \beta_{\max}) = (0.3, 2.0)$. Rank-based selection avoids dependence on task-specific score scales. When the search is progressing ($s_t \approx 0$), sampling concentrates on leading elites; as stagnation increases, the distribution flattens to expose lower-ranked regions of the archive. The selected parent is inserted into one of six uniformly sampled variation prompts: general mutation, focused single-weakness repair, mechanism replacement, analysis-targeted mutation, structural crossover, and component-level crossover. For crossover, the second parent is drawn from a different niche whenever possible. Every proposal is executed, considered for archive admission under the rule above, and recorded in $\mathcal{H}_t$ regardless of whether it is admitted.

Navigator: sparse, state-dependent intervention. At each Navigator checkpoint, the intervention mode is determined by the stagnation signal:

$$m_t = \begin{cases} \text{SYNTHESIS}, & s_t \leq \gamma_1, \\ \text{SURGICAL}, & \gamma_1 < s_t \leq \gamma_2, \\ \text{REFRAME}, & s_t > \gamma_2, \end{cases} \quad (\gamma_1, \gamma_2) = (0.4, 0.7). \quad (4)$$

When progress remains steady, SYNTHESIS combines complementary mechanisms from competitive niches into a coherent program. Under moderate stagnation, SURGICAL applies a targeted structural correction to the current champion. Under stronger stagnation, REFRAME introduces a strategy family absent from the current population. The intervention therefore shifts from recombination to targeted repair and, when necessary, to a more fundamental change in search strategy.

The Navigator never observes the full population. Instead, it receives a two-resolution view consisting of complete code for a small number of high-ranking niche incumbents—three anchors for SYNTHESIS, the champion alone for SURGICAL, and two anchors for REFRAME—together with description–score summaries of five additional niches. This design bounds context while exposing behaviorally distinct alternatives without encouraging verbatim imitation (Appendix E.2). The Navigator outputs a complete candidate program and its strategy description. The candidate is evaluated immediately; if valid, the Speculators subsequently generate $J=5$ nearby variants, allowing a single expensive intervention to seed several low-cost follow-up evaluations.

Advisor: consolidating evidence into guidance. Every $\Delta_A=50$ evaluations, the Advisor converts recent trajectory statistics and accumulated failure information into a compact prescriptive note. From $\mathcal{H}_t$ and the archive, it identifies *working* niches whose incumbents improved during the latest interval, *saturated* niches that received substantial evaluation effort without incumbent movement, and a persistent memory of recurring execution-failure signatures with representative evaluator messages (Appendix E.3). Conditioned on these signals and its previous note, the Advisor rewrites guidance of at most 300 words describing which mechanisms should be preserved, which directions should be de-emphasized, and which recurring errors should be avoided. The note thus acts as a rolling, revisable memory rather than an ever-growing transcript. To reduce feedback-induced herding, it is independently included in each Speculator prompt with probability $\rho_A=0.35$, leaving the majority of proposals to explore without explicit Advisor guidance.

The search terminates when the prescribed budget is exhausted and returns the highest-scoring program in the final archive.

4 EXPERIMENTS

We organize our experiments around three research questions:

RQ1: Does SPECEVO outperform strong LLM-driven evolutionary methods under matched wall-clock, API-cost, and evaluation budgets?

RQ2: Does the *speculate-then-consolidate* workflow outperform existing cost-efficient strategies that already exploit the asymmetry between lightweight and frontier models?

Table 1: Performance under a budget of 500 model calls. SPECEVO achieves the best performance on four of six tasks and ranks among the top two on all tasks, while using the frontier model for only 2.4% of calls.

Strategy	Circle Packing		Circle Rect.		Heil. Conv.		Heil. Tri.		MinMax 14-3		Signal		Frontier Model Calls (%)
	Avg	Best	Avg	Best	Avg	Best	Avg	Best	Avg	Best	Avg	Best	
RelayEvolve	2.521 $\pm$ 0.018	2.531	2.238 $\pm$ 0.020	2.249	0.022 $\pm$ 0.000	0.022	0.029 $\pm$ 0.001	0.030	0.221 $\pm$ 0.005	0.224	0.677 $\pm$ 0.012	0.684	89.0
All-cheap	2.265 $\pm$ 0.164	2.360	2.198 $\pm$ 0.018	2.219	0.020 $\pm$ 0.002	0.022	0.015 $\pm$ 0.007	0.019	0.218 $\pm$ 0.006	0.222	0.673 $\pm$ 0.005	0.678	0.0
Fixed-switch	2.500 $\pm$ 0.000	2.500	<u>2.264 $\pm$ 0.142</u>	2.346	0.021 $\pm$ 0.001	0.021	0.028 $\pm$ 0.004	0.032	0.229 $\pm$ 0.016	<u>0.238</u>	<u>0.725 $\pm$ 0.075</u>	<u>0.769</u>	50.0
Random	2.500 $\pm$ 0.000	2.500	1.762 $\pm$ 0.505	2.345	<u>0.024 $\pm$ 0.001</u>	0.024	0.023 $\pm$ 0.004	0.027	0.224 $\pm$ 0.000	0.224	0.659 $\pm$ 0.060	0.702	49.9
Bandit	2.471 $\pm$ 0.025	2.500	2.201 $\pm$ 0.087	2.251	<u>0.024 $\pm$ 0.002</u>	<u>0.026</u>	0.027 $\pm$ 0.002	0.029	<u>0.233 $\pm$ 0.004</u>	0.236	0.658 $\pm$ 0.032	0.695	50.2
Levi	<u>2.553 $\pm$ 0.045</u>	<u>2.618</u>	2.185 $\pm$ 0.121	<u>2.357</u>	0.023 $\pm$ 0.001	0.025	<u>0.033 $\pm$ 0.000</u>	<u>0.034</u>	0.236 $\pm$ 0.000	<u>0.238</u>	0.597 $\pm$ 0.036	0.623	3.2
SPECEVO (Ours)	2.593 $\pm$ 0.037	2.635	2.334 $\pm$ 0.029	2.361	0.026 $\pm$ 0.001	0.027	<u>0.031 $\pm$ 0.002</u>	<u>0.033</u>	0.239 $\pm$ 0.001	0.240	<u>0.709 $\pm$ 0.010</u>	<u>0.715</u>	2.4

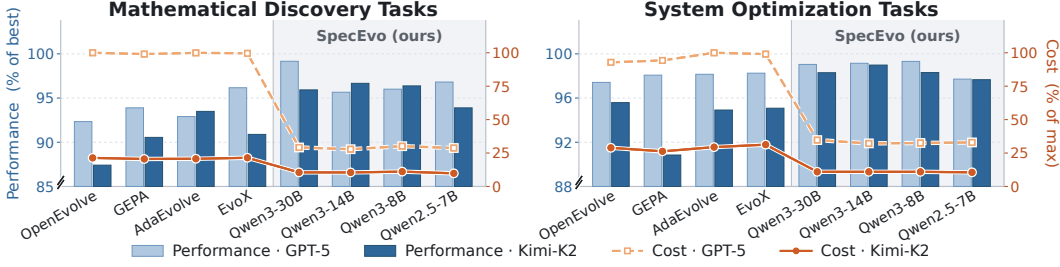


Figure 3: Performance under fixed three-hour wall-clock budget. SPECEVO attains the highest performance in both families while reducing API cost by roughly 50–70%, remains stable across model configurations.

RQ3: How do the Speculator, Navigator, and Advisor individually contribute to exploration diversity, strategic search progress, and failure reduction?

Benchmarks. We evaluate SPECEVO on 175 real-world scientific and algorithmic discovery tasks spanning four categories: mathematics (7 problems from AlphaEvolve Math (Novikov et al., 2025)), systems optimization (4 problems from ADRS (Cheng et al., 2025)), combinatorial optimization (36 problems from CO-Bench (Sun et al., 2026)), and scientific equation discovery (128 problems from LSR-Synth (Shojaee et al., 2025)). Detailed benchmark descriptions are provided in Appendix G.

Baselines and setups. For **RQ1**, we compare SPECEVO against strong LLM-driven evolutionary frameworks, including OpenEvolve (Sharma, 2025), GEPA (Agrawal et al., 2025), AdaEvolve (Cemri et al., 2026), and EvoX (Liu et al., 2026b). For **RQ2**, we compare against cost-efficient heterogeneous-model approaches, including LEVI (Tanveer, 2026) and RelayEvolve (Luo et al., 2026a). Following the evaluation design of RelayEvolve, we additionally include four controlled model-allocation strategies—All-cheap, Fixed-switch, Random, and Bandit—to isolate the effect of heterogeneous-model coordination; their definitions and implementation details are provided in Appendix D. We evaluate frontier proprietary and open-weight models: GPT-5 (OpenAI, 2025) and Kimi-K2 (Team et al., 2025), together with four Qwen models ranging from 7B to 30B parameters (Qwen Team, 2025), all accessed through OpenRouter (See Appendix G.1 for details). Experiments are conducted on a dual-socket server with two 16-core Intel Xeon Silver 4314 CPUs under three complementary resource constraints: a 3-hour wall-clock budget, a \$10 API-cost budget, and a fixed budget of 500 candidate evaluations. We report mean and best performance over three independent runs. Section 4.2 presents component ablations and hyperparameter sensitivity analyses. Full hyperparameters and metric definitions are provided in Appendices E and F, respectively.

4.1 MAIN RESULTS

RQ1: Overall effectiveness. SPECEVO remains consistently strong across domains, budgets, and model configurations, indicating that its gains arise from search organization rather than the strength of the backbone. Within the three-hour wall-clock budget, SPECEVO leads most mathematical settings and all systems settings at substantially lower API cost than frontier-heavy baselines, while also attaining the highest aggregate CO-Bench score—3.8% above the strongest competitor (Figures 3 and 4). SPECEVO maintains strong performance across multiple frontier and lightweight backbones, whereas competing methods degrade markedly as the backbone configuration changes

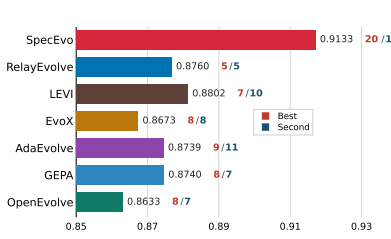


Figure 4: Performance comparison on CO-Bench, averaged across 36 problems.

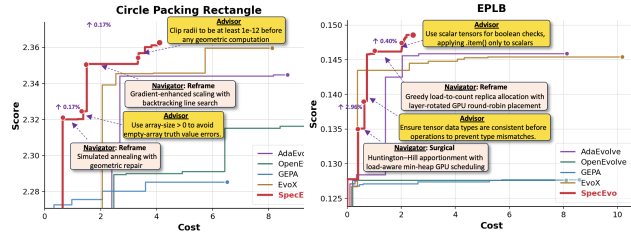


Figure 5: Cost-quality tradeoff with GPT-5. Navigator and Advisor accelerate search and help escape local optima.

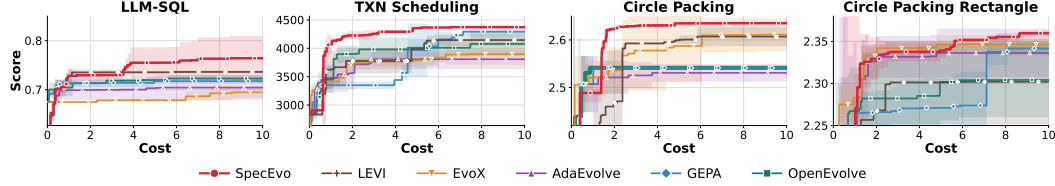


Figure 6: Performance comparison under \$10 API cost.

(up to a 20% drop with Qwen3-30B backbone; Figure 1a). With GPT-5 as the frontier model, all small-model variants attain best score of **0.240** on MinMax 14-3 and maximum score of **26.256** on PRISM. Moreover, while several baselines degrade noticeably when frontier backbone is changed from GPT-5 to Kimi-K2, SPECEVO remains competitive across configurations (Appendix H.2). See Appendices H.1- H.3 for detailed results and cost breakdowns. Figure 5 further shows that SPECEVO reaches stronger solutions earlier and at substantially lower API cost, whereas other baselines improve more slowly and plateau at lower quality despite greater expenditure.

Under the \$10 API budget, SPECEVO outperforms the strongest baseline by shifting routine exploration to inexpensive Speculators and reserving sparse, history-informed frontier calls for strategic intervention (Figure 6), with the largest improvement observed on LLM-SQL (5.12% gain). Importantly, its gains persist when search coverage is controlled: with all methods restricted to 500 evaluations on LSR-Synth, SPECEVO achieves the lowest NMSE across all eight domain-split settings and strong OOD performance, while competing methods require approximately 43%–65% more evaluations budgets to reach comparable quality (Table 2 and Figure 1b). The consistent advantage on OOD data further suggests that SPECEVO discovers more generalizable functional structure rather than merely fitting observed samples. Further analyses are provided in Appendix H.4

RQ2: Heterogeneous model coordination. SPECEVO’s advantage stems not merely from combining inexpensive and frontier models, but from coordinating their roles using accumulated search evidence. Under the common 500-call budget in Table 1, All-cheap lacks frontier-level strategic redirection, while Fixed-switch, Random, and Bandit do not jointly adapt model allocation and intervention type to trajectory-wide progress, failures, and saturation. Existing heterogeneous systems only partially close this loop: RelayEvo performs a one-way handoff that terminates lightweight exploration, whereas LEVI reserves the frontier model primarily for predefined paradigm shifts. In contrast, SPECEVO invokes the Navigator sparsely at fixed checkpoints, uses stagnation to select among synthesis, surgical refinement, and reframing, expands each intervention through inexpensive follow-up variants, and returns consolidated evidence to the Speculators through the Advisor. This coordination achieves the best result on most tasks and never ranks below second while using the frontier model for only 2.4% of calls, compared with 89.0% for RelayEvo. Since LEVI uses a similarly small frontier-call share, SPECEVO’s advantage suggests that performance depends on how scarce frontier reasoning is integrated into search, rather than on model-price asymmetry alone.

RQ3: Search dynamics. The search traces reveal how three roles form a complementary feedback loop. Speculator counters lightweight models’ tendency to recycle familiar structures by combining stagnation-adaptive parent sampling with specialized variation prompts. As a result, SPECEVO

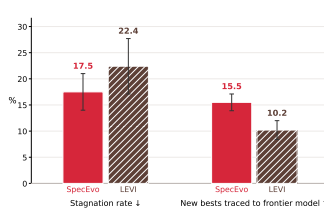


Figure 7: Stagnation rates and frontier-model contributions.

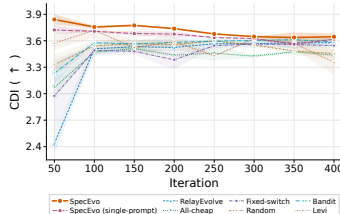


Figure 8: Diversity analysis of population.

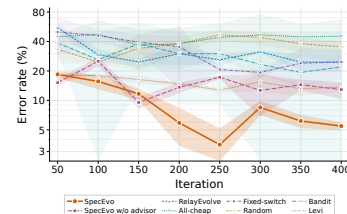


Figure 9: Execution error rates with and without the Advisor.

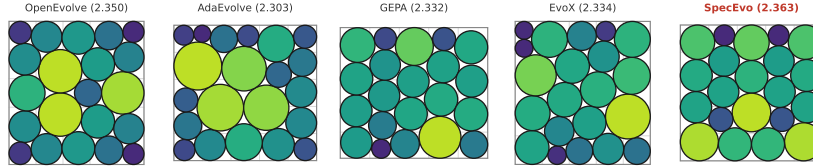


Figure 10: Best solutions generated by different methods for circle packing rectangle.

maintains higher code diversity than both its single-prompt ablation and LEVI, with the gap widening later in the search (Figure 8, (Dat et al., 2025)). Its archive retains qualitatively distinct strategy families, including analytic template-based constructions, penalty-annealed search, and neighbor-query-accelerated local refinement (see Appendix J for qualitative examples). Diversity alone, however, does not ensure continued progress once the population converges on a productive but exhausted region. At sparse checkpoints, Navigator uses stagnation to shift among synthesis, surgical refinement, and reframing, enabling strategic redirection rather than routine refinement. Accordingly, SPECEVO exhibits lower stagnation than LEVI and traces a larger share of new bests to Navigator proposals or their lightweight descendants (Figure 7); Figure 5 provides concrete examples of these interventions redirecting stalled searches toward better solutions. The Advisor closes the feedback loop at the execution level by converting recurring failures, including invalid indexing, undefined names, dtype mismatches, constraint violations, and timeouts, into reusable guidance. This reduces execution errors to approximately 5%–8%, compared with 13%–25% for LEVI (Figure 9), while stochastic injection leaves most prompts unguided to preserve independent exploration.

4.2 ABLATION STUDIES

We ablate the speculation and consolidation components of SPECEVO’s STC workflow (Table 3). For speculation, we remove AST features or description embeddings from the archive descriptor, disable periodic reclustering, or replace the six specialized prompts with a single general mutation operator. For consolidation, we remove the Navigator or replace its adaptive mode selection with random selection or a fixed mode (Synthesis, Surgical, or Reframe). We also remove the Advisor or replace its evolving guidance with fixed advice. We further assess sensitivity to the number of archive clusters, Navigator and Advisor intervals, and Speculator count (Figures 11, 12).

Component contributions. Table 3 supports two complementary sources of improvement: diverse exploration and consolidation that adapts to the search trajectory. Structural and semantic descriptors preserve distinct implementations and strategies, while reclustering keeps niches aligned with the evolving population. Specialized Speculator prompts sustain complementary search moves; collapsing them into a single operator produces the largest exploration-side degradation, particularly on Heilbronn Triangle. The consolidation ablations highlight the importance of conditioning feedback on search state. Random Navigator modes underperform on both tasks, while the strongest fixed mode differs across tasks, supporting the value of matching intervention scope to stagnation. Fixed guidance likewise underperforms adaptive Advisor updates and even yields lower mean performance than removing the Advisor on Heilbronn Triangle. This suggests that static advice can become counterproductive as the search evolves, motivating continual revision based on productive mechanisms, saturated directions, and recurring failures.

Table 2: Performance on LSR-Synth under 500 evaluations. SPECeVO outperforms other baselines, which require 43%–65% more evaluations to match its performance.

Domain	Split	OE	GEPA	AdaEvo	EvoX	RelayEvo	Levi	SPECeVO
Chemistry	ID	5.27E-06	1.72E-05	2.48E-06	6.87E-06	2.59E-06	5.80E-06	4.57E-07
	OOD	4.99E-02	2.69E-01	6.05E-02	8.68E-02	1.30E-01	1.17E-01	1.68E-02
Biology	ID	6.33E-05	8.79E-05	9.92E-06	3.79E-05	1.01E-05	3.87E-06	3.34E-07
	OOD	7.48E-01	1.92E+00	1.28E-01	5.83E-01	1.56E-01	1.96E-02	3.74E-03
Materials	ID	5.92E-08	1.86E-07	6.32E-07	9.05E-08	5.79E-08	5.73E-07	4.84E-08
	OOD	2.44E-06	7.57E-05	1.18E-04	1.63E-05	6.72E-06	8.36E-06	1.68E-06
Physics	ID	3.29E-04	1.53E-03	2.79E-04	6.66E-04	1.81E-04	1.38E-05	2.92E-06
	OOD	6.93E-03	2.82E-02	1.32E-02	5.32E-02	3.30E-03	2.23E-03	4.75E-04
Evals. to Beat SPECeVO		764 (+52.8%)	824 (+64.8%)	715 (+43.0%)	780 (+56.0%)	768 (+53.6%)	752 (+50.4%)	–

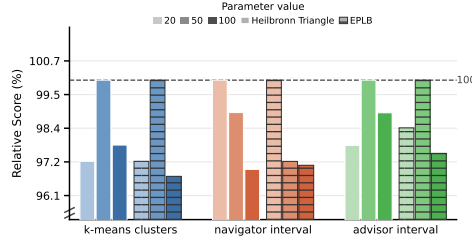


Figure 11: Sensitivity to the number of k -means clusters and Navigator/Advisor intervals.

Table 3: Ablation study on representative tasks.

Ablation Setting	Circle Packing Rect ($\uparrow$)	Heilbronn Triangle ($\uparrow$)
SpecEvo (full)	2.3634 $\pm$ 0.0007	0.0361 $\pm$ 0.0000
w/o AST features	2.3544 $\pm$ 0.0049 $\downarrow$ 0.38%	0.0333 $\pm$ 0.0018 $\downarrow$ 7.76%
w/o des embedding	2.3604 $\pm$ 0.0032 $\downarrow$ 0.13%	0.0338 $\pm$ 0.0017 $\downarrow$ 6.37%
w/o recluster	2.3243 $\pm$ 0.0187 $\downarrow$ 1.65%	0.0343 $\pm$ 0.0010 $\downarrow$ 4.99%
Single operator prompt	2.3122 $\pm$ 0.0799 $\downarrow$ 2.17%	0.0315 $\pm$ 0.0001 $\downarrow$ 12.74%
w/o Navigator	2.3102 $\pm$ 0.0398 $\downarrow$ 2.25%	0.0327 $\pm$ 0.0021 $\downarrow$ 9.42%
Random mode	2.3018 $\pm$ 0.0311 $\downarrow$ 2.60%	0.0334 $\pm$ 0.0011 $\downarrow$ 7.47%
Synthesis mode only	2.2994 $\pm$ 0.0309 $\downarrow$ 2.71%	0.0358 $\pm$ 0.0001 $\downarrow$ 0.83%
Surgical mode only	2.3518 $\pm$ 0.0085 $\downarrow$ 0.49%	0.0316 $\pm$ 0.0019 $\downarrow$ 12.47%
Reframe mode only	2.3169 $\pm$ 0.0197 $\downarrow$ 1.97%	0.0333 $\pm$ 0.0001 $\downarrow$ 7.76%
w/o Advisor	2.2731 $\pm$ 0.0018 $\downarrow$ 3.82%	0.0339 $\pm$ 0.0009 $\downarrow$ 6.09%
Fixed guidance	2.2814 $\pm$ 0.0016 $\downarrow$ 3.46%	0.0331 $\pm$ 0.0005 $\downarrow$ 8.31%

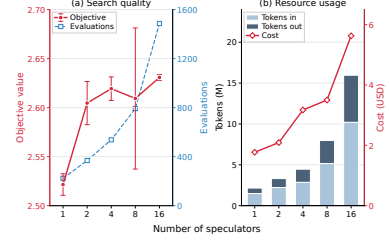


Figure 12: Effect of the number of Speculators on search quality and resource usage.

Case studies. Figure 5 makes this division of labor concrete. On Circle Packing Rectangle, routine refinement initially converges around similar geometric constructions; the Navigator therefore uses Reframe to introduce simulated annealing with geometric repair and later gradient-enhanced scaling with backtracking line search. The Advisor complements these strategic shifts with targeted safeguards for empty arrays and near-zero radii, allowing subsequent Speculator variants to explore the new neighborhoods without repeatedly failing on numerical edge cases (Figure 10). A similar pattern appears on EPLB: Surgical first strengthens the incumbent through Huntington–Hill apportionment and load-aware heap scheduling, while Reframe later replaces the saturated placement logic with load-to-count allocation and rotated round-robin assignment. Between these interventions, the Advisor prevents recurring dtype and scalar-boolean errors.

Hyperparameter sensitivity. Figure 11 favors 50 clusters, balancing strategy coverage with sufficient refinement within each niche. The preferred Navigator and Advisor intervals, 20 and 50 evaluations respectively, reflect different informational needs: strategic intervention benefits from timely responses to stagnation, whereas reusable guidance requires enough observations to distinguish recurring patterns from isolated outcomes. Figure 12 reveals diminishing returns from parallel exploration on Circle Packing. Scaling from one to four Speculators improves both quality and throughput. Beyond four, throughput continues to grow, but quality gains become marginal while token usage and API cost rise sharply. We therefore use four Speculators to capture most of the benefit of parallel exploration at moderate cost.

5 CONCLUSION

We introduced SPECeVO, a resource-aware framework that closes the feedback loop between low-cost exploration and sparse frontier-model reasoning in LLM-driven evolutionary search. Its *speculate-then-consolidate* workflow enables lightweight Speculators to explore in parallel, while accumulated successes, failures, and saturated directions continually refine persistent guidance and determine how the Navigator intervenes. Across 175 scientific and algorithmic discovery tasks, SPECeVO consistently matches or outperforms strong baselines across wall-clock, API-cost, and evaluation budgets while reducing API expenditure by 50%–80%. Its gains under matched evaluation budgets and across backbone configurations show that efficient discovery depends not on using stronger models more often, but on using search evidence to decide when and how they should intervene.

REFERENCES

- Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, et al. Automix: Automatically mixing language models. *Advances in Neural Information Processing Systems*, 37:131000–131034, 2024.
- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. In *First Workshop on Foundations of Reasoning in Language Models*, 2025.
- Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, et al. Adaevolve: Adaptive llm driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023a.
- Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023b.
- Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, et al. Barbarians at the gate: How ai is upending systems research, 2025. URL <https://arxiv.org/abs/2510.06189>, 5(4), 2025.
- Pham Vu Tuan Dat, Long Doan, and Huynh Thi Thanh Binh. Hsevo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 26931–26938, 2025.
- Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks Lakshmanan, and Ahmed H Awadallah. Hybrid llm: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations*, volume 2024, pp. 41348–41366, 2024.
- Ha Minh Hieu, Hung Phan, Tung Duy Doan, Tung Dao, Cong Dao Tran, and Huynh Thi Thanh Binh. Pareto-grid-guided large language models for fast and high-quality heuristics design in multi-objective combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 36964–36972, 2026.
- Yiqian Huang, Shiqi Zhang, Tianyuan Jin, and Xiaokui Xiao. Scope: Cost-efficient model selection for compound ai systems under quality constraints. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, KDD ’26, pp. 1922–1931, New York, NY, USA, 2026. Association for Computing Machinery. ISBN 9798400722592. doi: 10.1145/3770855.3818067. URL <https://doi.org/10.1145/3770855.3818067>.
- William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabrício Olivetti de França, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason H Moore. Contemporary symbolic regression methods and their relative performance. *arXiv preprint arXiv:2107.14351*, 2021.
- Robert Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. In *International Conference on Learning Representations*, volume 2026, pp. 74026–74078, 2026.
- Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. In *Handbook of evolutionary machine learning*, pp. 331–366. Springer, 2023.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International conference on machine learning*, pp. 19274–19286. PMLR, 2023.
- Yuchen Li, Handing Wang, and Yaochu Jin. Evosr-llm: Evolutionary symbolic regression guided by large language models. *IEEE Transactions on Evolutionary Computation*, 2026.

- Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, Li Zhang, Zhongqi Li, and Yuchi Ma. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv e-prints*, pp. arXiv-2404, 2024a.
- Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024b.
- Fei Liu, Yiming Yao, Ping Guo, Zhiyuan Yang, Xi Lin, Zhe Zhao, Xialiang Tong, Kun Mao, Zhichao Lu, Zhenkun Wang, et al. A systematic survey on large language models for algorithm design. *ACM Computing Surveys*, 58(8):1–32, 2026a.
- Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z Pan, et al. Evox: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026b.
- Ximeng Liu, Qianlong Wang, Yingming Mao, Annan Li, Yatao Li, Shizhen Zhao, Jianmin Wu, Dawei Yin, and Dou Shen. Janus: An algorithm-evaluator co-evolution framework for llm-driven discovery under expensive evaluation budgets. *arXiv preprint arXiv:2608.08189*, 2026c.
- Chris Lu, Samuel Holt, Claudio Fanconi, Alex J Chan, Jakob Foerster, Mihaela van der Schaar, and Robert T Lange. Discovering preference optimization algorithms with and for large language models. *Advances in Neural Information Processing Systems*, 37:86528–86573, 2024a.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024b.
- Sichun Luo, Yi Huang, Guanzhi Deng, Haibo Wang, Haochen Luo, Lei Li, Zefa Hu, Junlan Feng, and Qi Liu. Relay, don’t route: Adaptive population handoff for cost-efficient llm-driven evolution. *arXiv preprint arXiv:2608.05651*, 2026a.
- Sichun Luo, Yi Huang, Haochen Luo, Fengyuan Liu, Guanzhi Deng, Lei Li, Qinghua Yao, Zefa Hu, Junlan Feng, and Qi Liu. Seaevo: Advancing algorithm discovery with strategy space evolution. *arXiv preprint arXiv:2604.24372*, 2026b.
- Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Jim Fan, et al. Eureka: Human-level reward design via coding large language models. In *International conference on learning Representations*, volume 2024, pp. 26516–26560, 2024.
- Yoshitomo Matsubara, Naoya Chiba, Ryo Igarashi, and Yoshitaka Ushiku. Rethinking symbolic regression datasets and benchmarks for scientific discovery. *arXiv preprint arXiv:2206.10540*, 2022.
- Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- Alexander Novikov, Ngân Vŭ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, Mohammed Kadous, and Ion Stoica. Routellm: Learning to route llms from preference data. In *International Conference on Learning Representations*, volume 2025, pp. 34433–34448, 2025.
- OpenAI. GPT-5 System Card. <https://openai.com/index/gpt-5-system-card/>, August 2025.
- Steven T Piantadosi. Zipf’s word frequency law in natural language: A critical review and future directions. *Psychonomic bulletin & review*, 21(5):1112–1130, 2014.

- Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, et al. Coral: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint arXiv:2604.01658*, 2026.
- Qwen Team. Qwen3: Think deeper, act faster. <https://qwenlm.github.io/blog/qwen3/>, April 2025.
- Pretam Ray, Pratik Prabhanjan Brahma, Zicheng Liu, and Emad Barsoum. Adaptevolve: Improving efficiency of evolutionary ai agents through adaptive model selection. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 40625–40633, 2026.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco JR Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025. URL <https://github.com/algorithmicsuperintelligence/openevolve>.
- Parshin Shojaei, Ngoc-Hieu Nguyen, Kazem Meidani, Amir Barati Farimani, Khoa D Doan, and Chandan K Reddy. Llm-srbench: A new benchmark for scientific equation discovery with large language models. *arXiv preprint arXiv:2504.10415*, 2025.
- Weiwei Sun, Shengyu Feng, Shanda Li, and Yiming Yang. Co-bench: Benchmarking language model agents in algorithm search for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 33126–33134, 2026.
- Temoor Tanveer. Levi: Stronger search architectures can substitute for larger llms in evolutionary search. *arXiv preprint arXiv:2605.09764*, 2026.
- Kimi Team, Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- Niki Van Stein and Thomas Bäck. Llamea: A large language model evolutionary algorithm for automatically generating metaheuristics. *IEEE Transactions on Evolutionary Computation*, 29(2):331–345, 2024.
- Yiping Wang, Shao-Rong Su, Zhiyuan Zeng, Eva Xu, Liliang Ren, Xinyu Yang, Zeyi Huang, Xuehai He, Luyao Ma, Baolin Peng, et al. Thetaevolve: Test-time learning on open problems. *arXiv preprint arXiv:2511.23473*, 2025.
- Xingyu Wu, Sheng-hao Wu, Jibin Wu, Liang Feng, and Kay Chen Tan. Evolutionary computation in the era of large language model: Survey and roadmap. *IEEE Transactions on Evolutionary Computation*, 29(2):534–554, 2024.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.
- Minghao Yan, Bo Peng, Benjamin Coleman, Ziqi Chen, Zhouhang Xie, Shuo Chen, Zhankui He, Naveen Sachdeva, Isabella Ye, Weili Wang, et al. Pacevolve: Enabling long-horizon progress-aware consistent evolution. *arXiv preprint arXiv:2601.10657*, 2026.
- Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608, 2024.

SPECEVO: Speculative Evolution with Large Language Models for Cost-Efficient Scientific Discovery

(Appendix)

This appendix provides supplementary material supporting the methodology, evaluation, and analysis presented in the main paper. Appendices A and B discuss the limitations of SPECEVO and disclose our use of AI for language editing. Appendices C and D extend the related-work discussion and document the baselines and common evaluation protocol. Appendix E provides the complete algorithmic specification of SPECEVO, including the Speculator loop and the Navigator and Advisor interventions, while Appendix F defines the metrics used to analyze search dynamics. Appendices G and H describe the benchmark suites and provide detailed experimental results, including API cost and token usage breakdowns. Appendix I presents the complete prompt templates for all three roles, and Appendix J analyzes generated programs, recurring failure modes, algorithmic mechanisms, and the guidance consolidated by the Advisor on Circle Packing Rectangle and EPLB. For reproducibility, the programs generated by SPECEVO are also included in the accompanying code repository.

CONTENTS

A	Limitations	14
B	AI Use Statement	14
C	Related Works	14
D	Baseline Details	16
E	Algorithm Details	17
E.1	The complete SPECEVO workflow	17
E.2	Navigator intervention	18
E.3	Advisor intervention	20
F	Metric Details	21
F.1	Code Diversity Index (CDI)	21
F.2	Stagnation rate	21
F.3	New bests traced to the frontier model	22
G	Benchmark Details	22
G.1	Mathematical Optimization Benchmarks	22
G.2	Systems Optimization Benchmarks	22
G.3	CO-Bench	22
G.4	LSR-Synth	23
H	Additional Experimental Results	25
H.1	Breakdown of API Cost and token usage	25
H.2	Detailed Results for Mathematical Discovery and Systems Optimization Tasks	25
H.3	Detailed Results for Each CO-Bench Problem Category	27
H.4	Detailed Results on LSR-Synth	28
I	Prompt Templates	28
I.1	Speculator	28
I.2	Navigator	35
I.3	Advisor	36
J	Qualitative Analysis	37
J.1	Circle Packing Rectangle	37
J.2	EPLB	40

A LIMITATIONS

Dependence on parallel computing infrastructure. The wall-clock efficiency of SPECEVO partly depends on the amount of parallelism available to the Speculator pool. Our experiments use $W = 4$ concurrent Speculators on a dual-socket server with two 16-core Intel Xeon Silver 4314 processors. Consequently, the number of candidates explored within a fixed wall-clock budget may change with the worker count, CPU capacity, memory bandwidth, network latency, and API rate limits. Although our fixed API-cost and fixed evaluation-budget experiments provide complementary controls, they do not make the wall-clock results hardware-independent. The reported cost reductions should therefore be interpreted primarily as savings in LLM API expenditure under the specified computing environment, rather than reductions in total computational cost.

Compute-intensive candidate evaluation. The benchmarks considered in this work use relatively lightweight evaluators: candidate programs can generally be executed and scored on CPUs without repeatedly training large neural networks, running long simulations, or conducting physical experiments. This setting allows multiple Speculators to evaluate proposals concurrently at modest local-compute cost. For tasks whose evaluation requires GPU training, large-scale inference, expensive simulation, or real-world experimentation, increasing the number of parallel workers may instead multiply accelerator usage, memory requirements, and the cost of unsuccessful trials. Moreover, a fixed number of evaluations does not represent a fixed compute budget when candidate evaluations have heterogeneous costs. Evaluating SPECEVO in such settings is an important direction for future work. A more comprehensive study should jointly control API expenditure, CPU- or GPU-hours, peak memory, energy consumption, wall-clock time, and the number of evaluated candidates. It would also be valuable to adapt the worker count and the frequencies of Navigator and Advisor intervention dynamically according to evaluator cost, available hardware, and observed search progress.

B AI USE STATEMENT

In this work, we used LLMs solely to assist with language editing and to improve the clarity and readability of the manuscript. We did not use generative AI tools for any other research tasks requiring disclosure, and the remaining disclosure categories are not applicable to this work. All AI-assisted edits were carefully reviewed by the authors. We take full responsibility for the final content of this paper, including any text produced with the aid of generative AI.

C RELATED WORKS

LLM-Guided Evolutionary Search. LLM-guided evolutionary search combines language models as semantic variation operators with executable evaluators and population-based optimization. FunSearch established this paradigm by evolving programs for open mathematical problems (Shojaee et al., 2025; Romera-Paredes et al., 2024), while ELM combined pretrained code models with quality-diversity search (Lehman et al., 2023). Subsequent methods specialize the evolutionary loop in different ways: EoH jointly evolves natural-language ideas and executable heuristics (Liu et al., 2024b); ReEvo derives reflective guidance from performance comparisons (Ye et al., 2024); LLaMEA evolves optimization algorithms through iterative evaluation (Van Stein & Bäck, 2024); HSEvo explicitly balances diversity and convergence (Dat et al., 2025); and MPaGE extends heuristic evolution to multiple objectives (Hieu et al., 2026). Related systems apply LLM-guided search to reward design (Ma et al., 2024), preference optimization (Lu et al., 2024a), and scientific equation discovery (Li et al., 2026; Shojaee et al., 2025). More general evolutionary coding agents include AlphaEvolve (Novikov et al., 2025), OpenEvolve (Sharma, 2025), and ShinkaEvolve (Lange et al., 2026), while GEPA evolves prompts and natural-language rules through reflection and Pareto selection (Agrawal et al., 2025). Recent work further improves the representation and control of search: SeaEvo organizes candidates into persistent semantic strategy families (Luo et al., 2026b); EvoX evolves the search strategy itself (Liu et al., 2026b); AdaEvolve adapts exploration intensity, resource allocation across islands, and meta-level guidance (Cemri et al., 2026); and CORAL delegates search decisions to autonomous peer agents communicating through shared memory (Qu et al., 2026).

Despite these advances, existing systems largely lack the hierarchical and resource-asymmetric division of labor characteristic of real research laboratories, where routine experimentation and high-level scientific judgment are assigned to researchers with different levels of expertise and invoked at different frequencies. As a result, strong models may be used for both ordinary candidate generation and consequential strategic reasoning, increasing inference cost and latency and making effective search difficult to reproduce under limited computational resources. SPECEVO instead emulates the collaborative workflow of a resource-constrained research laboratory through three differentiated roles: lightweight Speculators act as junior researchers that conduct frequent, broad experimentation; the Advisor acts as a postdoctoral researcher that periodically consolidates recurring successes and failures into actionable guidance; and the frontier-model Navigator acts as the principal investigator, intervening sparsely to make population-level strategic decisions. By coordinating models with different capabilities, responsibilities, and intervention frequencies throughout the search, SPECEVO reserves expensive reasoning for decisions with broader downstream impact while delegating routine exploration to inexpensive models, substantially reducing both API cost and wall-clock time.

Cost-Aware LLM Routing and Evolutionary Search Allocation. Cost-aware LLM systems reduce inference expense by selecting among models with different capabilities. FrugalGPT constructs cascades that progressively escalate difficult queries (Chen et al., 2023b), AutoMix uses self-verification to decide whether a small-model response requires augmentation by a stronger model (Aggarwal et al., 2024), and RouteLLM learns routing policies from human preference data (Ong et al., 2025). These methods focus on independent inference requests, whereas evolutionary search must allocate resources across a sequence of interdependent population updates. ShinkaEvolve addresses this setting with a bandit that selects mutation models according to their observed performance (Lange et al., 2026). AdaptEvolve instead runs a small model first and uses token-level confidence statistics with an online decision-tree router to determine whether the mutation should be regenerated by a larger model (Ray et al., 2026). AdaEvolve allocates computation across concurrent evolutionary processes according to their recent progress and introduces meta-level reasoning to adjust search behavior under stagnation (Cemri et al., 2026). RelayEvolve shifts allocation from individual mutations to entire search phases: a cheap model develops multiple trajectories in short blocks, Relay Gain measures the marginal quality and coverage of their candidate bank, and saturation of this signal triggers a one-time handoff of a curated seed population to strong-model evolution (Luo et al., 2026a). LEVI combines three complementary mechanisms: bootstrapped CVT-MAP-Elites for diversity preservation, role-aware routing that assigns local refinements to smaller models and paradigm shifts to a stronger model, and rank-preserving proxy benchmarks that reduce evaluation overhead (Tanveer, 2026).

These approaches optimize different components of resource allocation but retain important structural restrictions. ShinkaEvolve and AdaptEvolve select among models at the mutation level, leaving them functionally interchangeable; AdaptEvolve additionally requires token-level probabilities that are unavailable from many proprietary APIs. AdaEvolve adapts the amount and direction of search effort rather than organizing heterogeneous models into persistent functional roles. RelayEvolve captures population-level state more explicitly, but information flows through a single cheap-to-strong transition: once handoff occurs, lightweight exploration ends, and the strong phase inherits selected candidates rather than consolidated knowledge about discarded attempts, recurring failures, or exhausted trajectories. LEVI provides a clearer division between local and structural mutations, but its stronger model is assigned primarily to a predefined paradigm-shift operation, without a dedicated component that continually aggregates trajectory-wide evidence for subsequent small-model search. SPECEVO instead treats model heterogeneity as a persistent division of labor throughout evolution. Its Speculators perform high-throughput variation, its Advisor repeatedly converts successes, failures, and niche-level progress into reusable guidance, and its Navigator examines the population at sparse checkpoints to choose among synthesis, surgical refinement, and reframing. Thus, rather than optimizing only model selection, phase transition, or mutation type, SPECEVO coordinates candidate generation, experience consolidation, and strategic intervention within the same evolutionary process.

D BASELINE DETAILS

Common evaluation protocol. We evaluate all baselines using their native search logic and publicly released implementations. We introduce only the task adapters required to expose the same problem specification, editable program scaffold, execution environment, and evaluator to every method; we otherwise preserve each method’s population management, prompting strategy, and default hyperparameters without task-specific tuning. For the single-backbone baselines—OpenEvolve, GEPA, AdaEvolve, and EvoX—all generation, reflection, and meta-level calls use the designated frontier backbone. For heterogeneous-model methods, the same frontier model is paired with Qwen3-30B-A3B-Instruct-2507 as the lightweight model. The 500-call RQ2 comparison uses Kimi-K2-Thinking and Qwen3-30B-A3B-Instruct-2507 as this shared model pair.

All initialization, mutation, repair, reflection, meta-guidance, and paradigm-shift calls are included in API-cost and wall-clock accounting. API expenditure is computed from the actual input and output tokens of each call using the prices in Table 9. Under an evaluation budget, every candidate submitted to the evaluator consumes one evaluation, including programs that fail to parse, violate constraints, or time out; auxiliary calls that do not produce a candidate do not increment this counter but still contribute to API cost and elapsed time. Under the 500-call RQ2 budget, every LLM invocation counts toward the common limit. Each method returns the highest-scoring valid program discovered before its stopping condition is reached.

OpenEvolve. OpenEvolve (Sharma, 2025) implements asynchronous island-based program evolution with a MAP-Elites-style database. Candidate parents are selected from a mixture of high-performing and behaviorally diverse programs, mutated using evaluator feedback, and periodically exchanged across islands. We retain its native archive, migration, and prompt construction mechanisms, while assigning the selected frontier backbone to all LLM calls.

GEPA. We use the inference-time code-optimization configuration of GEPA (Agrawal et al., 2025), treating the candidate program as the textual artifact being optimized. GEPA converts execution traces and evaluator diagnostics into natural-language reflection, uses the resulting analysis to propose targeted mutations, and maintains complementary candidates through Pareto-aware selection and merging. The same frontier backbone is used for both reflection and candidate generation.

AdaEvolve. AdaEvolve (Cemri et al., 2026) derives an accumulated-improvement signal from the search trajectory and uses it at three levels: adjusting exploration intensity within each island, allocating evaluations across islands through a bandit scheduler, and generating new high-level tactics when global progress stalls. We use the authors’ released implementation and default adaptive schedules, with the designated frontier backbone serving both mutation and meta-guidance calls.

EvoX. EvoX (Liu et al., 2026b) jointly evolves candidate programs and the strategy used to generate them. Its inner loop evaluates programs under the current search strategy, while its outer loop periodically revises parent selection and variation behavior according to the progress achieved during the preceding search window. We preserve this two-level evolution and use the same frontier backbone for solution generation and search-strategy evolution.

LEVI. LEVI (Tanveer, 2026) first generates diverse seeds to calibrate a CVT-MAP-Elites archive and then applies role-aware model routing. Qwen3-30B performs ordinary local refinements, whereas the frontier model is reserved for periodically triggered paradigm shifts conditioned on representatives from distinct archive regions. We retain its native archive descriptors, trigger policy, lightweight follow-up variants, and proxy-benchmark construction where applicable. All reported final programs are re-evaluated using the complete task evaluator.

RelayEvolve. RelayEvolve (Luo et al., 2026a) begins with multiple lightweight-model trajectories whose search blocks are allocated by a Grow–Deepen bandit. Relay Gain measures the marginal quality and coverage contributed to an online candidate bank and triggers handoff when further cheap-model exploration saturates. The terminal candidate pool is then curated using the method’s quality–diversity objective, and the selected population initializes a shared frontier-model search using the remaining budget. We retain the default strong-model budget reserve and all handoff hyperparameters. Once handoff occurs, lightweight-model exploration does not resume.

Controlled model-allocation strategies. To isolate the effect of model coordination from the mere availability of two differently priced models, we additionally implement four controls on a shared asynchronous evolutionary backend. They use identical parent selection, population updates, mutation prompts, evaluator feedback, and context limits, differing only in how the lightweight and frontier models are assigned. For a total call budget B , these strategies are defined as follows:

- **All-cheap** assigns every call to the lightweight model.
- **Fixed-switch** uses the lightweight model for the first $B/2$ calls and then performs a one-time switch to the frontier model for the remaining calls, carrying the current population forward without additional curation.
- **Random** independently selects the lightweight or frontier model with probability 0.5 for every call.
- **Bandit** treats the two models as arms of a UCB1 scheduler. After initializing both arms, model selection balances their observed mean rewards with an exploration bonus. The reward is the non-negative improvement in the global best score produced by the selected model; invalid or non-improving proposals receive zero reward.

These controls deliberately use a common mutation role and therefore do not introduce Navigator-, Advisor-, paradigm-shift-, or handoff-specific prompts. An additional All-strong control is omitted because the frontier-only baselines above already represent the corresponding high-cost allocation regime.

E ALGORITHM DETAILS

We set the temperatures to 0.5, 0.5, and 0.8 for the *synthesis*, *surgical*, and *reframe* modes, respectively, and to 0.7 and 0.4 for the Speculator and Advisor. We set $H_g = 100$, $H_l = 20$, $\beta_{\min} = 0.3$, $\beta_{\max} = 2.0$, $\gamma_1 = 0.4$, and $\gamma_2 = 0.7$. We recluster the archive every $\Delta_C = 50$ completed evaluations, setting $\Delta_C = \Delta_A$ for consistency with the Advisor update interval. After each Navigator call generates a set of hypotheses, the Speculator produces $J = 5$ variants for each hypothesis.

E.1 THE COMPLETE SPECEVO WORKFLOW

Algorithm 1 illustrates the overall workflow of SPECEVO. Initialization first uses a small number of frontier-model calls to obtain a diverse set of executable strategy families. Lightweight variants then expand these seeds into a sufficiently broad population for constructing the behavioral archive. After initialization, W asynchronous Speculators perform nearly all candidate generation, while the two consolidation routines operate on independent evaluation schedules.

Speculator step. Let the archive programs be ordered by decreasing score, with r_i denoting the rank of p_i . At each worker step, the parent distribution and its stagnation-dependent exponent are:

$$\Pr(p_i \mid \mathcal{A}_t) = \frac{r_i^{-\beta_t}}{\sum_{j=1}^{|\mathcal{A}_t|} r_j^{-\beta_t}}, \quad \beta_t = \beta_{\min} + (1 - s_t)(\beta_{\max} - \beta_{\min}). \quad (5)$$

For crossover, the second parent is drawn from a different niche whenever possible. Additional inspirations are sampled without replacement from the same rank distribution after excluding the parent(s), but only their descriptions and scores are shown. This distinction is intentional: parent code supplies a concrete object to modify, whereas description-only inspirations broaden the idea set without encouraging verbatim copying or inflating the prompt.

The six variation prompts are general mutation, focused single-weakness repair, mechanism replacement, analysis-targeted mutation, structural crossover, and component-level crossover. They are sampled uniformly, so each prompt is used with equal probability during Speculator generation.

Asynchrony and checkpoint cadence. The evaluation index t counts completed candidate evaluations rather than optimizer “generations.” Because Speculators run asynchronously, multiple evaluations may complete between periodic checks, so a trigger can occur slightly after its nominal

Algorithm 1 SpecEvo: asynchronous speculate–then–consolidate search

Require: Task specification $\mathcal{T}$, evaluator F , budget $\mathcal{B}$, number of Speculator workers W , Speculator $\mathcal{M}_S$, Navigator $\mathcal{M}_N$, Advisor $\mathcal{M}_A$, number of initialization seeds M , number of variants per seed V , number of archive clusters K , Navigator interval Δ_N , Advisor interval Δ_A , reclustering interval Δ_C , Advisor-guidance probability ρ_A

Ensure: $\arg \max_{p \in \mathcal{A}_t} F(p)$

```

1:  $t \leftarrow 0$  ▷ completed evaluations
2:  $\mathcal{A}_t, \mathcal{H}_t, \mathcal{S}_0 \leftarrow \emptyset; \ell \leftarrow \emptyset$  ▷ Frontier-seeded initialization

3: for  $i = 1, \dots, M$  do
4:    $(p_i^0, d_i^0) \sim \mathcal{M}_N(\cdot \mid \mathcal{T}, \{(p_j^0, d_j^0, y_j^0)\}_{j < i})$ 
5:    $(y_i^0, \mathcal{A}_t, \mathcal{H}_t, t) \leftarrow \text{EVALUATEUPDATE}(p_i^0, d_i^0, F, \mathcal{A}_t, \mathcal{H}_t, t)$ 
6:   if  $p_i^0$  is executable then
7:      $\mathcal{S}_0 \leftarrow \mathcal{S}_0 \cup \{(p_i^0, d_i^0, y_i^0)\}$  ▷ Lightweight diversification

8: for all  $(p_i^0, d_i^0, y_i^0) \in \mathcal{S}_0$  in parallel over up to  $W$  workers do
9:   for  $j = 1, \dots, V$  do
10:     $I_{i,j} \leftarrow \text{INSPIRATIONS}(\mathcal{S}_0, i)$ 
11:     $(p_{i,j}^0, d_{i,j}^0) \sim \mathcal{M}_S(\cdot \mid \mathcal{T}, p_i^0, I_{i,j})$ 
12:     $(\cdot, \mathcal{A}_t, \mathcal{H}_t, t) \leftarrow \text{EVALUATEUPDATE}(p_{i,j}^0, d_{i,j}^0, F, \mathcal{A}_t, \mathcal{H}_t, t)$ 
13:  $\mathcal{A}_t \leftarrow \text{RECLUSTER}(\mathcal{A}_t, K)$ 
14:  $t_N \leftarrow t; t_A \leftarrow t; t_C \leftarrow t$  ▷ Asynchronous speculate–then–consolidate search

15: while no stopping condition in  $\mathcal{B}$  is met do
16:    $s_t \leftarrow \text{STAGNATION}(\mathcal{H}_t)$ 
17:   for all idle Speculator workers do
18:      $(P_t, I_t, o_t) \leftarrow \text{SPECULATIONCONTEXT}(\mathcal{A}_t, s_t)$ 
19:      $\tilde{\ell} \leftarrow \ell$  with probability  $\rho_A$ ; otherwise  $\tilde{\ell} \leftarrow \emptyset$ 
20:     Dispatch  $(p, d) \sim \mathcal{M}_S(\cdot \mid \mathcal{T}, P_t, I_t, o_t, \tilde{\ell})$ 
21:      $(p, d) \leftarrow \text{AWAITNEXT}$ 
22:      $(\cdot, \mathcal{A}_t, \mathcal{H}_t, t) \leftarrow \text{EVALUATEUPDATE}(p, d, F, \mathcal{A}_t, \mathcal{H}_t, t)$ 
23:     if  $t - t_N \geq \Delta_N$  then
24:        $(\mathcal{A}_t, \mathcal{H}_t, t) \leftarrow \text{NAVIGATOR}(\mathcal{T}, F, \mathcal{A}_t, \mathcal{H}_t, t)$ 
25:        $t_N \leftarrow t$ 
26:     if  $t - t_A \geq \Delta_A$  then
27:        $\ell \leftarrow \text{ADVISOR}(\mathcal{T}, \mathcal{A}_t, \mathcal{H}_t, \ell, t_A, t)$ 
28:        $t_A \leftarrow t$ 
29:     if  $t - t_C \geq \Delta_C$  then
30:        $\mathcal{A}_t \leftarrow \text{RECLUSTER}(\mathcal{A}_t, K)$ 
31:        $t_C \leftarrow t$ 
32: return  $\arg \max_{p \in \mathcal{A}_t} F(p)$ 

```

interval. After initialization, the Navigator, Advisor, and archive reclustering maintain independent clocks t_N , t_A , and t_C , all reset to the current t . They are subsequently invoked whenever $t - t_N \geq \Delta_N$, $t - t_A \geq \Delta_A$, and $t - t_C \geq \Delta_C$, respectively. Thus, initialization does not consume any of these intervals, and each periodic operation proceeds independently according to its own evaluation cadence.

E.2 NAVIGATOR INTERVENTION

Algorithm 2 illustrates the overall workflow of a Navigator invocation.

Selecting anchors and inspirations. The Navigator receives a two-resolution view of the behavioral archive. We first collect the incumbent from each occupied niche and rank these niche elites by score. The highest-ranked elites are selected as *anchors*, for which the Navigator observes the complete program, strategy description, and score. The subsequent elites are used as *inspirations*, exposing only their descriptions and scores. Anchors and inspirations are disjoint within each invocation.

Algorithm 2 Navigator consolidation

Require: Task specification $\mathcal{T}$, evaluator F , archive $\mathcal{A}_t$, history $\mathcal{H}_t$, evaluation index t , Navigator $\mathcal{M}_N$, Speculator $\mathcal{M}_S$, stagnation thresholds γ_1, γ_2 , number of fan-out variants J

Ensure: $\mathcal{A}_t, \mathcal{H}_t, t$

```

1:  $(q_1, \dots, q_{K_t}) \leftarrow \text{RANKEDINCUMBENTS}(\mathcal{A}_t)$  ▷ one elite per occupied niche
2:  $s_t \leftarrow \text{STAGNATION}(\mathcal{H}_t)$ 
3: if  $s_t \leq \gamma_1$  then
4:    $m_t \leftarrow \text{SYNTHESIS}; a \leftarrow 3$ 
5: else if  $s_t \leq \gamma_2$  then
6:    $m_t \leftarrow \text{SURGICAL}; a \leftarrow 1$ 
7: else
8:    $m_t \leftarrow \text{REFRAME}; a \leftarrow 2$ 
9:  $k \leftarrow 5$  ▷ Construct a two-resolution population view

10:  $A_t^{(m)} \leftarrow \{(q_i, d(q_i), F(q_i)) : 1 \leq i \leq \min(a, K_t)\}$ 
11:  $I_t^{(m)} \leftarrow \{(d(q_i), F(q_i)) : a < i \leq \min(a + k, K_t)\}$ 
12:  $V_t^{(m)} \leftarrow \text{POPULATIONVIEW}(A_t^{(m)}, I_t^{(m)}, s_t, \mathcal{H}_t)$  ▷ Frontier-model intervention

13:  $(p_N, d_N) \sim \mathcal{M}_N(\cdot \mid \mathcal{T}, V_t^{(m)}, m_t)$ 
14:  $y_N \leftarrow F(p_N); t \leftarrow t + 1$ 
15: if  $y_N$  is invalid then
16:    $\mathcal{H}_t \leftarrow \text{RECORDFAILURE}(\mathcal{H}_t, p_N, d_N)$ 
17:   return  $(\mathcal{A}_t, \mathcal{H}_t, t)$ 
18:  $(\mathcal{A}_t, \mathcal{H}_t) \leftarrow \text{UPDATE}(\mathcal{A}_t, \mathcal{H}_t, p_N, d_N, y_N)$  ▷ Lightweight refinement of the Navigator proposal

19: for  $j = 1, \dots, J$  in parallel do
20:    $(p_{N,j}, d_{N,j}) \sim \mathcal{M}_S(\cdot \mid \mathcal{T}, p_N, d_N)$ 
21:    $(\cdot, \mathcal{A}_t, \mathcal{H}_t, t) \leftarrow \text{EVALUATEUPDATE}(p_{N,j}, d_{N,j}, F, \mathcal{A}_t, \mathcal{H}_t, t)$ 
22: return  $(\mathcal{A}_t, \mathcal{H}_t, t)$ 

```

Mode	Full-code anchors	Description-only inspirations	Requested move
SYNTHESIS	top 3 niche elites	next 5 niche elites	combine mechanisms across anchors
SURGICAL	global champion only	next 5 niche elites	one local structural correction
REFRAME	top 2 niche elites	next 5 niche elites	introduce an absent strategy family

Table 4: Mode-specific context provided to the Navigator.

This separation controls both context usage and the scope of intervention. Full-code anchors provide the concrete implementation details required for precise reasoning and modification, but including too many complete programs would substantially increase context and encourage direct imitation of existing solutions. Description-only inspirations instead expose a broader set of mechanisms and search directions without overwhelming the prompt, allowing the Navigator to reason about alternatives beyond the strongest incumbents. Ranking niche elites rather than the entire population further ensures that this context reflects behaviorally distinct solutions rather than many near-duplicate high-scoring programs.

The amount of full-code context is adapted to the intervention mode, as summarized in Table 4. SYNTHESIS uses multiple strong anchors because combining complementary mechanisms requires access to their implementations. SURGICAL focuses only on the global champion, since its objective is to preserve the current solution while correcting a localized weakness. REFRAME uses two strong anchors to identify assumptions shared by successful solutions, while relying on broader description-level context to seek a qualitatively different strategy family. Consequently, the three modes progressively change the *scope* of the Navigator intervention rather than merely varying the amount of context.

Algorithm 3 Advisor checkpoint

Require: Task specification $\mathcal{T}$, archive $\mathcal{A}_t$, history $\mathcal{H}_t$, evaluation index t , previous Advisor checkpoint t_A , previous advice ℓ , failure memory $\mathcal{R}$, Advisor $\mathcal{M}_A$

Ensure: Updated advice ℓ and failure memory $\mathcal{R}$

- 1: $(a_k, m_k)_k \leftarrow \text{REGION SIGNALS}(\mathcal{A}_t, \mathcal{H}_t, t_A, t)$ ▷ activity and movement
- 2: $D_t^{\text{work}} \leftarrow \text{TOPSCORE}(\{\mathcal{A}_t(k) : m_k = 1\}, 4)$
- 3: $D_t^{\text{sat}} \leftarrow \text{TOPACTIVITY}(\{k : m_k = 0 \wedge a_k > 0\}, 3)$
- 4: $E_t^{\text{new}} \leftarrow \text{NEWERRORS}(\mathcal{H}_t, t_A, t)$
- 5: $\mathcal{R} \leftarrow \text{UPDATEERRORMEMORY}(\mathcal{R}, E_t^{\text{new}})$
- 6: $E_t \leftarrow \text{MOSTFREQUENT}(\mathcal{R}, 8)$
- 7: $X_t \leftarrow (D_t^{\text{work}}, D_t^{\text{sat}}, E_t, \ell, y_t^{\text{best}}, s_t)$
- 8: $\ell \sim \mathcal{M}_A(\cdot \mid \mathcal{T}, X_t)$ ▷ distill shared guidance
- 9: **return** $(\ell, \mathcal{R})$

Together with the stagnation-dependent mode selection, this design shifts the Navigator from recombining promising mechanisms to targeted repair and, under stronger stagnation, to more substantial changes in the current search strategy.

E.3 ADVISOR INTERVENTION

Algorithm 3 summarizes the overall workflow of an Advisor intervention, from extracting niche-level search signals and recurrent failures to distilling guidance for subsequent Speculator generations.

Tracking niche activity and progress. At each Advisor checkpoint, we inspect the evaluations completed since the previous checkpoint. For every occupied niche k , we maintain two signals: an *activity count* a_k , equal to the number of evaluated candidates assigned to that niche during the interval, and a *movement indicator* m_k , which records whether the niche incumbent was replaced during the same interval. Importantly, the activity count includes unsuccessful candidates. Hence, a niche can receive substantial search effort without producing a better incumbent.

These signals distinguish productive search from saturation without requiring a task-specific improvement threshold. A niche with recent incumbent movement is treated as actively improving, whereas a niche with high activity but no movement is considered saturated. This distinction is particularly useful across our heterogeneous benchmarks, where the scale of score improvements varies substantially between tasks.

At each checkpoint, the Advisor constructs four complementary population views:

- D_t^{work} : up to four highest-scoring niches whose incumbents moved during the current interval;
- D_t^{sat} : up to three most active niches that received evaluations but showed no incumbent movement.

Together, these views separate current solution quality from recent search progress and exploration coverage. In particular, a high-scoring niche need not be improving, while a lower-scoring niche may reveal a productive direction that has only recently emerged.

Failure memory. Population statistics alone do not capture repeated implementation failures. We therefore maintain a persistent error memory $\mathcal{R}$ across Advisor checkpoints. Similar evaluator errors are normalized into compact signatures by lowercasing the message, removing numbers and punctuation, collapsing whitespace, and retaining a short prefix. This groups failures that share the same underlying cause while avoiding a benchmark-specific error taxonomy. The memory stores at most 24 signatures and tracks their frequencies; the eight most frequent signatures, together with representative messages, are provided to the Advisor. This allows recurrent failure modes to influence future guidance even when they span multiple Advisor intervals.

Advisor output and guidance injection. The Advisor converts the population views, recurrent failures, and previous advice into a compact note with three fields. `WORKING` summarizes mechanisms associated with leading or recently improving niches; `SATURATED` identifies directions that continue to consume evaluations without moving their incumbents; and `AVOID` highlights recurrent anti-patterns supported by the failure memory.

We use temperature 0.4 and a 400-token generation limit, with the resulting note constrained to fewer than 300 words. The note is not imposed on every worker. Instead, each subsequent Speculator receives it independently with probability $\rho_A = 0.35$. This stochastic injection allows useful population-level lessons to propagate through the search while preserving a substantial fraction of independent exploration.

F METRIC DETAILS

F.1 CODE DIVERSITY INDEX (CDI)

CDI (Dat et al., 2025) measures how evenly the population’s code representations are separated along a minimum spanning tree (MST). At checkpoint t , let $\mathcal{Q}_t = \{p_1, \dots, p_n\}$ be the complete archive snapshot. We first canonicalize each source string. For each Python program, we parse its abstract syntax tree, remove comments and module/function/class docstrings, and emit a deterministically formatted program. If parsing fails, we fall back to whitespace-normalized source. Let

$$x_i = f_{\text{emb}}(\text{canon}(\text{code}(p_i))) \in \mathbb{R}^d \quad (6)$$

be the embedding of the resulting code. We use `text-embedding-3-small` for all methods and checkpoints.

Construct the complete undirected graph on the n embeddings with Euclidean edge weight

$$d_{ij} = \|x_i - x_j\|_2. \quad (7)$$

Let $\mathcal{E}_t^{\text{MST}}$ be the edge set of a minimum spanning tree, and let $\mathcal{E}_t^+ = \{e \in \mathcal{E}_t^{\text{MST}} : d_e > 0\}$ remove zero-length duplicate edges. Normalize its positive edge lengths as

$$\pi_e = \frac{d_e}{\sum_{e' \in \mathcal{E}_t^+} d_{e'}}. \quad (8)$$

The reported Code Diversity Index is the Shannon entropy

$$\text{CDI}(\mathcal{Q}_t) = - \sum_{e \in \mathcal{E}_t^+} \pi_e \log \pi_e, \quad (9)$$

where $\log$ is the natural logarithm. If there is no positive MST edge (e.g., $n \leq 1$ or all embeddings coincide), CDI is defined as zero.

CDI is high when separation is distributed across many MST edges instead of being dominated by one bridge between otherwise collapsed groups. Since the edge lengths are normalized, uniform rescaling of every embedding distance does not change CDI; the metric captures the distribution of population separations, not an absolute diameter. With $m = |\mathcal{E}_t^+| \geq 1$ positive edges, $0 \leq \text{CDI} \leq \log m \leq \log(n-1)$, with the upper bound attained when the positive MST edges have equal length. Because this upper bound depends on population size, comparisons should use checkpoints with comparable archive sizes. Our computation keeps duplicate programs by default and uses the archive exactly as stored at the checkpoint.

F.2 STAGNATION RATE

The reported *stagnation rate* is a post-hoc, run-level statistic that asks what fraction of the full evaluation budget was consumed by the single longest global-best plateau.

For a run of T evaluations, define the new-best index set

$$\mathcal{I}_{\text{best}} = \left\{ t \in \{1, \dots, T\} : y_t > \max_{1 \leq j < t} y_j \right\} = \{b_1 < \dots < b_m\}. \quad (10)$$

Set the sentinels $b_0 = 0$ and $b_{m+1} = T + 1$. The number of consecutive non-new-best evaluations between two adjacent events is $g_i = b_{i+1} - b_i - 1$. This convention includes the prefix before the first new best and the suffix after the last new best. The run-level metric is

$$\text{StagRate} = 100 \times \frac{\max_{0 \leq i \leq m} g_i}{T} \quad (\%). \quad (11)$$

F.3 NEW BESTS TRACED TO THE FRONTIER MODEL

This metric measures how often observed progress can be attributed to the frontier model. In LEVI, the frontier model is invoked through `paradigm_shift`, while in SPECEVO it serves as the Navigator. For each new-best event b_i in Eq. equation 10, define

$$z(b_i) = \begin{cases} 1, & \text{if } b_i \text{ is generated by the frontier model, or by a small model} \\ & \text{with at least one frontier-generated parent,} \\ 0, & \text{otherwise.} \end{cases} \quad (12)$$

The run-level rate is

$$\text{FrontierTrace} = 100 \times \frac{\sum_{i=1}^m z(b_i)}{m} \quad (\%), \quad (13)$$

where m is the number of new-best events. Thus, `FrontierTrace` measures the fraction of improvements that are either directly produced by the frontier model or arise from lightweight refinement of its programs. Initialization is excluded, and the metric is reported as missing when $m = 0$. Since it counts improvement events rather than their magnitude, it should be interpreted together with final performance and API cost.

G BENCHMARK DETAILS

We provide additional details about the benchmarks used in our evaluation. Unless otherwise stated, a candidate solution is represented as an executable program and evaluated by a task-specific evaluator. The evolutionary methods only observe the resulting scalar score and diagnostic feedback, without access to hidden test instances or ground-truth solutions.

G.1 MATHEMATICAL OPTIMIZATION BENCHMARKS

We consider seven executable mathematical optimization tasks covering geometric packing, extremal point configurations, and signal processing. The geometric tasks follow the formulations used in AlphaEvolve (Novikov et al., 2025). Each candidate program constructs or refines a feasible solution, and the evaluator returns a scalar higher-is-better objective value. The exact configurations used in our experiments are summarized in Table 5.

G.2 SYSTEMS OPTIMIZATION BENCHMARKS

We use four real-world systems optimization tasks from the AI-Driven Research for Systems benchmark suite (Cheng et al., 2025). Although some underlying objectives are naturally formulated as minimization problems, the task evaluators convert them into scalar scores for which a larger value indicates a better candidate in all reported experiments. Table 6 summarizes the four tasks included in our evaluation.

G.3 CO-BENCH

CO-Bench (Sun et al., 2026) evaluates the ability of language-model agents to design complete algorithms for combinatorial optimization rather than directly predicting solutions to individual instances. It contains 36 real-world optimization problems spanning eight categories and a broad range of problem sizes and constraint structures. Most instances are curated from OR-Library and related operations-research collections.

For each problem, the benchmark provides the agent with a natural language problem specification, a starter `solve` interface, and a development set. A hidden evaluation function checks feasibility and

Table 5: Mathematical optimization benchmarks used in our evaluation.

Problem	Configuration	Objective and task definition
Circle Packing (Square)	$N = 26$	Pack N pairwise-disjoint circles inside a unit square and maximize the sum of their radii.
Circle Packing (Rectangle)	$N = 21$	Pack N pairwise-disjoint circles inside a rectangle with perimeter 4 and maximize the sum of their radii.
Heilbronn Problem (Triangle)	$N = 11$	Place N points on or inside a triangle of unit area so as to maximize the minimum area of any triangle formed by three selected points.
Heilbronn Problem (Convex Region)	$N = 13$	Place N points on or inside a convex region of unit area so as to maximize the minimum area of any triangle formed by three selected points.
MinMax Distance	$(N, d) = (16, 2)$	Find N points in $\mathbb{R}^d$ that maximize the ratio between the minimum and maximum pairwise Euclidean distances.
MinMax Distance	$(N, d) = (14, 3)$	Optimize the same MinMax-distance objective for 14 points in three-dimensional Euclidean space.
Signal Processing		Optimize a continuous signal processing objective under noisy evaluation, where candidate programs transform an input signal and are scored by an external evaluator.

measures solution quality. The test instances and evaluator implementation are unavailable during search, reducing direct overfitting to the final evaluation data.

Let $h(x; p)$ denote the objective value, including any constraint penalty, of solution x on instance p , and let h_p^* denote the corresponding optimal or best-known value. CO-Bench normalizes solution quality as

$$s(x, p) = \frac{h(x; p)}{h_p^*}. \quad (14)$$

A score of 1 indicates performance matching the optimal or best-known solution. Programs that fail, time out, or return an infeasible solution receive a score of 0. The score for each problem is averaged over its test instances, and the overall CO-Bench score is the unweighted mean across all 36 problems:

$$S(A) = \frac{1}{36} \sum_{c=1}^{36} \frac{1}{|\mathcal{P}_c|} \sum_{p \in \mathcal{P}_c} s(A(p), p), \quad (15)$$

where A is a candidate algorithm and $\mathcal{P}_c$ is the test set of problem c . Due to the three-hour wall-clock budget, we sample at most 50 instances for each problem and split them into training and test sets using a 7:3 ratio, enabling a comprehensive comparison between SPECEVO and the baselines under a consistent computational budget.

Table 7 lists all problem families included in our evaluation.

G.4 LSR-SYNTH

We use LSR-Synth from LLM-SRBench (Shojaee et al., 2025), a benchmark designed to evaluate LLM-based scientific equation discovery. Unlike traditional symbolic regression benchmarks that often contain well-established canonical equations (La Cava et al., 2021; Matsubara et al., 2022), LSR-Synth combines known functional terms with novel synthetic terms. This construction reduces the likelihood that models can solve the tasks through memorization or direct recall of equations encountered during pre-training.

Each problem provides a scientific context, descriptions of the relevant variables, and an observational dataset. The objective is to discover a mathematical expression that accurately captures the underlying relationship between the input variables and the target quantity. LSR-Synth covers four scientific domains: chemistry, biology, physics, and material science.

Table 6: Systems optimization benchmarks used in our evaluation.

Task	Objective	Problem description
Expert Parallelism Load Balancer (EPLB)	Balance expert-parallel workloads across GPUs	Given expert load statistics for a mixture-of-experts model, determine the number and placement of expert replicas across physical GPU slots. The objective is to reduce computational skew and stragglers while satisfying placement constraints and preserving inference throughput.
Model Placement (PRISM)	Optimize model-to-GPU placement cost	Assign multiple models or model components to a collection of GPUs under memory and capacity constraints. The placement should minimize global execution cost or peak resource pressure while maintaining feasibility and service-level requirements.
LLM-SQL	Maximize prefix-cache efficiency	Reorder table rows, columns, or fields so that serialized SQL inputs share longer prefixes and obtain higher prefix-cache reuse. The transformation must preserve the original table semantics and query results.
Transaction Scheduling (TXN)	Minimize transaction-execution makespan	Determine an execution order for dependent or conflicting database transactions. The scheduler seeks to reduce total completion time while respecting dependencies and avoiding excessive conflicts, aborts, or waiting time.

Table 7: The 36 combinatorial optimization problems in CO-Bench, grouped into eight categories.

Category	#	Included problems
Packing	9	Bin packing; Multi-Demand Multidimensional Knapsack problem; Multidimensional knapsack problem; Container loading; Container loading with weight restrictions; Packing unequal circles; Packing unequal rectangles and squares number / area.
Cutting	4	Assortment problem; Constrained / unconstrained guillotine cutting; Constrained non-guillotine cutting.
Facility location	4	Capacitated / Uncapacitated warehouse location; Capacitated / Uncapacitated p-median problem.
Scheduling	7	Aircraft landing; Crew scheduling; Common due date scheduling; Flow shop scheduling; Hybrid Reentrant Shop Scheduling; Job shop scheduling; Open shop scheduling.
Routing	3	Traveling salesman problem; Period vehicle routing problem; Resource constrained shortest path.
Assignment	2	Constrained / unconstrained assignment.
Tree	2	Euclidean Steiner; Corporate structuring.
Graph and set	5	Maximal Independent Set; Graph colouring; Equitable partitioning; Set partitioning; Set covering.
Total	36	

We use all 128 problems from LSR-Synth, comprising 36 chemistry problems, 24 biology problems, 43 physics problems, and 25 material science problems.

Table 8: Statistics of the LSR-Synth problems used in our evaluation.

	Chemistry	Biology	Physics	Material	Overall
# Problems	36	24	43	25	128
Avg. Datapoints (k)	5.0	5.0	5.0	5.0	5.0
Avg. Variants	3.0	3.0	3.9	3.0	3.3

During the search process, candidate equations are ranked using

$$\text{Score} = \log_{10} \left(\frac{1}{1 + \text{NMSE}_{\text{train}}} \right),$$

where $\text{NMSE}_{\text{train}}$ is computed on the training data. For the final evaluation, we directly report NMSE on both the in-distribution (ID) test set, whose variable ranges follow those seen during training, and the out-of-distribution (OOD) test set, which contains values outside the training ranges to assess extrapolation ability.

Detailed statistics are reported in Table 8.

H ADDITIONAL EXPERIMENTAL RESULTS

H.1 BREAKDOWN OF API COST AND TOKEN USAGE

Table 9 reports the OpenRouter list prices of all models used in our experiments, and Figure 13 shows the corresponding input and output token usage on six representative tasks with GPT-5 as the frontier-model backbone. The figure makes an important point: SPECEVO often consumes more total tokens than several baselines, and in some cases even more than LEVI, because it delegates a large volume of routine search, mutation, and refinement to lightweight Speculators. However, higher token usage does not imply higher cost. The reason is that most of these additional tokens are generated by inexpensive small models, while costly frontier-model calls are reserved for infrequent Navigator interventions. Given the large price gap between models—for example, GPT-5 output tokens cost \$10.00 per million, compared with only \$0.30 for Qwen3-30B-A3B-Instruct-2507—shifting the bulk of the search workload to lightweight models yields substantial savings even when the total token count is larger.

Tables 10 and 13 provide the task-level API expenditure under the fixed three-hour wall-clock budget. Excluding LEVI, SPECEVO reduces API cost by approximately 49.9%–70.7% across the four benchmark–backbone settings. LEVI is treated separately because it also adopts a heterogeneous model configuration, using a lightweight model for routine search and invoking the frontier model only for occasional `paradigm_shift` operations; hence, it already benefits from the same basic price asymmetry between lightweight and frontier models. The comparison with LEVI therefore more directly reflects differences in how scarce frontier-model reasoning is allocated, rather than savings from model substitution alone.

H.2 DETAILED RESULTS FOR MATHEMATICAL DISCOVERY AND SYSTEMS OPTIMIZATION TASKS

The following results disaggregate the aggregate trends reported in Figure 3 across individual tasks and backbone configurations. Beyond comparing final performance, we examine whether the gains persist when either the frontier model or the lightweight Speculator is changed, thereby separating the contribution of the search architecture from that of any particular model.

Mathematical discovery. Across both frontier backbones, SPECEVO attains the strongest or tied-best single-run result on six of the seven tasks and the leading or tied mean performance on five. Its advantage is most consistent on the Heilbronn and MinMax problems, whose landscapes contain multiple structurally distinct solution families and densely populated local optima. The adaptive archive preserves these alternatives, while the Navigator can synthesize promising mechanisms or reframe the search once lightweight refinement becomes saturated. Consequently, even smaller

Table 9: OpenRouter API pricing for the models used in our experiments, reported in USD per one million tokens (accessed August 1, 2026).

Model	Price per 1M Tokens (USD)	
	Input	Output
Frontier Models		
GPT-5	1.25	10.00
Kimi-K2-Thinking	0.60	2.50
Small Models		
Qwen3-30B-A3B-Instruct-2507	0.10	0.30
Qwen3-14B	0.10	0.22
Qwen3-8B	0.11	0.45
Qwen2.5-7B-Instruct	0.10	0.20
Embedding Model		
Text Embedding 3 Small	0.02	

Table 10: API cost in USD under the fixed three-hour wall-clock budget on mathematical discovery tasks. Each entry reports the mean and standard deviation over three independent runs. Lower is better.

Strategy	Circle Packing	Circle Rect.	Heil. Conv.	Heil. Tri.	MinMax 16-2	MinMax 14-3	Signal
<i>Backbone: GPT-5</i>							
OpenEvolve	7.65 ± 0.40	8.65 ± 1.01	8.76 ± 2.48	8.53 ± 0.02	8.31 ± 0.28	7.12 ± 0.86	8.53 ± 0.64
GEPA	8.41 ± 0.10	5.20 ± 1.14	8.37 ± 1.04	9.36 ± 1.71	7.82 ± 1.41	8.09 ± 0.55	9.74 ± 1.75
AdaEvolve	8.83 ± 0.31	6.68 ± 1.29	8.62 ± 0.06	8.93 ± 0.86	7.96 ± 0.26	8.58 ± 0.01	7.91 ± 0.91
EvoX	7.66 ± 0.50	7.33 ± 0.68	9.18 ± 0.22	9.12 ± 1.58	8.15 ± 0.56	7.17 ± 0.67	8.72 ± 2.05
Levi	3.40 ± 1.23	2.62 ± 1.05	1.60 ± 0.25	1.33 ± 0.19	1.80 ± 0.17	2.45 ± 0.75	3.64 ± 0.12
SpecEvo	3.37 ± 1.61	2.61 ± 1.29	1.62 ± 0.35	1.32 ± 0.18	1.65 ± 0.06	2.33 ± 0.86	3.88 ± 0.01
<i>Backbone: KIMI-K2</i>							
OpenEvolve	1.98 ± 0.20	1.63 ± 0.19	1.64 ± 0.16	1.63 ± 0.36	1.61 ± 0.21	1.52 ± 0.19	2.28 ± 0.28
GEPA	1.95 ± 0.10	1.69 ± 0.05	1.45 ± 0.16	1.45 ± 0.11	1.62 ± 0.04	1.49 ± 0.23	2.18 ± 0.26
AdaEvolve	1.67 ± 0.16	1.79 ± 0.31	1.59 ± 0.12	1.49 ± 0.24	1.63 ± 0.31	1.52 ± 0.09	2.23 ± 0.06
EvoX	1.76 ± 0.15	1.67 ± 0.22	1.62 ± 0.16	1.75 ± 0.22	1.55 ± 0.14	1.66 ± 0.39	2.33 ± 0.05
Levi	0.79 ± 0.12	0.77 ± 0.21	0.45 ± 0.15	0.65 ± 0.30	1.12 ± 0.49	1.04 ± 0.18	1.39 ± 0.20
SpecEvo	0.83 ± 0.33	0.83 ± 0.22	0.42 ± 0.21	0.50 ± 0.27	1.05 ± 0.27	1.02 ± 0.26	1.41 ± 0.14

Speculators frequently match the strongest configurations on these tasks. Signal Processing is the main exception: its noisy evaluation makes short-term progress less reliable, producing greater variation across lightweight backbones and allowing conventional baselines to remain competitive.

Systems optimization. The gains are more uniform on systems tasks, where at least one SPECEVO configuration achieves the best or tied-best mean performance on every problem under both GPT-5 and Kimi-K2. Importantly, no single lightweight backbone dominates throughout: Qwen3-30B performs strongest on EPLB and LLM-SQL with GPT-5, Qwen3-14B leads Transaction Scheduling, and Qwen3-8B becomes strongest on Transaction Scheduling with Kimi-K2. Even Qwen2.5-7B reaches the performance ceiling on PRISM. This non-monotonic relationship with model size suggests that the results are not explained simply by using a more capable Speculator. Instead, the closed-loop workflow allows inexpensive models with different strengths to conduct high-volume refinement, while population-level evidence determines when strategic redirection is required.

Robustness and cost efficiency. This division of labor also reduces sensitivity to the frontier backbone. Several frontier-only baselines degrade substantially when GPT-5 is replaced by Kimi-K2, particularly on PRISM, whereas SPECEVO remains close to the task ceiling because the frontier model influences the search through sparse, high-impact interventions rather than every mutation.

The same organization reduces API expenditure by 34%–85% on mathematical discovery and 51%–71% on systems optimization relative to the four frontier-heavy baselines. Moreover, SPECEVO generally improves upon LEVI at comparable cost, indicating that the remaining quality difference comes primarily from how scarce frontier reasoning and accumulated evidence are integrated into the search, rather than from model-price asymmetry alone.

Table 11: Performance comparison on mathematical discovery tasks. Red arrows indicate SPECEVO’s average API cost reduction relative to the four baselines, excluding Levi.

Strategy	Circle Packing		Circle Rect.		Heil. Conv.		Heil. Tri.		MinMax 16-2		MinMax 14-3		Signal	
	Avg	Best	Avg	Best	Avg	Best	Avg	Best	Avg	Best	Avg	Best	Avg	Best
<i>Backbone: GPT-5</i>														
OpenEvolve	2.541 ± 0.000	2.541	2.341 ± 0.022	2.354	0.022 ± 0.000	0.022	0.033 ± 0.002	0.036	<u>0.077</u> ± 0.000	<u>0.077</u>	0.229 ± 0.008	0.238	0.595 ± 0.005	0.598
GEPA	2.536 ± 0.010	2.541	2.313 ± 0.024	2.327	0.024 ± 0.004	<u>0.029</u>	0.032 ± 0.000	0.032	<u>0.077</u> ± 0.000	<u>0.077</u>	0.226 ± 0.002	0.227	0.593 ± 0.038	0.636
AdaEvolve	2.563 ± 0.037	2.606	2.296 ± 0.042	2.345	0.022 ± 0.001	0.023	0.029 ± 0.000	0.029	<u>0.077</u> ± 0.000	<u>0.077</u>	0.235 ± 0.003	0.237	0.733 ± 0.001	0.734
EvoX	2.521 ± 0.035	2.541	2.352 ± 0.007	2.360	0.027 ± 0.000	0.027	0.033 ± 0.001	0.034	0.078 ± 0.000	0.078	0.239 ± 0.000	0.239	0.688 ± 0.013	0.703
Levi	<u>2.593 ± 0.001</u>	2.595	2.301 ± 0.040	2.358	0.028 ± 0.001	<u>0.029</u>	0.034 ± 0.000	<u>0.035</u>	<u>0.077</u> ± 0.000	<u>0.077</u>	0.239 ± 0.000	<u>0.239</u>	0.593 ± 0.068	0.691
SPECEVO	↓59% (\$)		↓63% (\$)		↓81% (\$)		↓85% (\$)		↓80% (\$)		↓70% (\$)		↓56% (\$)	
Qwen3-30b	2.580 ± 0.041	2.627	2.338 ± 0.021	2.363	0.030 ± 0.000	0.030	0.034 ± 0.002	0.036	<u>0.077</u> ± 0.000	0.078	0.240 ± 0.000	0.240	<u>0.711 ± 0.012</u>	<u>0.717</u>
Qwen3-14b	2.629 ± 0.001	2.630	2.306 ± 0.012	2.342	<u>0.029 ± 0.001</u>	0.030	0.036 ± 0.000	0.036	<u>0.077</u> ± 0.000	0.078	0.240 ± 0.000	0.240	0.545 ± 0.012	0.552
Qwen3-8b	2.572 ± 0.055	2.636	2.351 ± 0.008	2.361	0.030 ± 0.000	0.030	<u>0.035 ± 0.001</u>	0.036	<u>0.077</u> ± 0.000	0.078	0.240 ± 0.000	0.240	0.550 ± 0.004	0.553
Qwen2.5-7b	2.517 ± 0.054	2.548	2.333 ± 0.052	2.363	0.025 ± 0.001	0.026	0.034 ± 0.002	0.036	0.078 ± 0.000	0.078	0.240 ± 0.000	0.240	0.708 ± 0.009	0.713
<i>Backbone: KIMI-K2</i>														
OpenEvolve	2.379 ± 0.109	2.500	2.184 ± 0.171	2.296	0.018 ± 0.001	0.019	0.026 ± 0.001	0.027	0.068 ± 0.011	0.075	0.213 ± 0.004	0.218	0.671 ± 0.045	0.709
GEPA	2.429 ± 0.062	2.481	2.167 ± 0.145	2.251	0.019 ± 0.002	0.022	0.021 ± 0.014	<u>0.035</u>	0.068 ± 0.011	0.075	0.211 ± 0.000	0.211	0.643 ± 0.020	0.658
AdaEvolve	2.595 ± 0.037	2.630	2.256 ± 0.005	2.261	0.025 ± 0.002	0.026	0.028 ± 0.005	0.031	0.074 ± 0.004	0.078	0.220 ± 0.008	0.227	0.614 ± 0.090	0.693
EvoX	2.561 ± 0.083	2.617	2.266 ± 0.063	2.336	0.024 ± 0.003	0.026	0.025 ± 0.003	0.027	0.073 ± 0.002	0.075	0.213 ± 0.004	0.218	<u>0.655 ± 0.031</u>	0.677
Levi	2.553 ± 0.045	2.618	2.185 ± 0.121	<u>2.357</u>	0.023 ± 0.001	0.025	<u>0.033 ± 0.000</u>	0.034	<u>0.076 ± 0.001</u>	<u>0.077</u>	0.237 ± 0.000	0.238	0.597 ± 0.036	0.623
SPECEVO	↓55% (\$)		↓51% (\$)		↓73% (\$)		↓68% (\$)		↓34% (\$)		↓34% (\$)		↓37% (\$)	
Qwen3-30b	<u>2.593 ± 0.037</u>	2.635	2.334 ± 0.029	2.361	0.026 ± 0.001	<u>0.027</u>	0.031 ± 0.002	0.033	0.078 ± 0.000	0.078	0.223 ± 0.012	0.236	0.629 ± 0.065	<u>0.703</u>
Qwen3-14b	2.539 ± 0.035	2.559	2.278 ± 0.007	2.282	0.030 ± 0.001	0.031	0.030 ± 0.007	0.034	0.078 ± 0.000	0.078	0.239 ± 0.001	0.240	0.624 ± 0.045	0.650
Qwen3-8b	2.584 ± 0.080	<u>2.631</u>	2.269 ± 0.146	2.353	<u>0.027 ± 0.000</u>	<u>0.027</u>	0.034 ± 0.003	0.036	0.078 ± 0.000	0.078	0.229 ± 0.009	<u>0.239</u>	0.613 ± 0.065	0.650
Qwen2.5-7b	2.489 ± 0.013	2.497	<u>2.315 ± 0.055</u>	2.347	0.023 ± 0.003	0.025	<u>0.033 ± 0.003</u>	<u>0.035</u>	0.073 ± 0.005	0.076	<u>0.238 ± 0.003</u>	0.240	0.615 ± 0.020	0.645

Table 12: Performance comparison on system optimization tasks.

Strategy	EPLB (↑)		LLM-SQL (↑)		Transaction (↑)		PRISM (↑)	
	Avg	Best	Avg	Best	Avg	Best	Avg	Best
<i>Backbone: GPT-5</i>								
OpenEvolve	0.1398 ± 0.011	0.1459	0.7152 ± 0.024	0.7289	4006.0 ± 65.47	4081.6	<u>26.242 ± 0.008</u>	<u>26.247</u>
GEPA	0.1394 ± 0.010	0.1453	0.7242 ± 0.006	0.7315	4078.2 ± 213.73	4201.6	26.235 ± 0.002	26.236
AdaEvolve	0.1462 ± 0.000	0.1464	<u>0.7287 ± 0.004</u>	0.7307	3958.7 ± 195.14	4184.1	26.256 ± 0.000	26.256
EvoX	0.1448 ± 0.001	0.1454	0.6936 ± 0.031	0.7298	3827.8 ± 354.60	4237.3	26.256 ± 0.000	26.256
Levi	0.1446 ± 0.0001	0.1449	0.7228 ± 0.0141	<u>0.7328</u>	3777.1 ± 180.37	4032.2	26.256 ± 0.000	26.256
SPECEVO	↓68% (\$)		↓71% (\$)		↓51% (\$)		↓57% (\$)	
Qwen3-30b	0.1472 ± 0.001	0.1485	0.7327 ± 0.003	0.7347	<u>4173.3 ± 71.01</u>	<u>4255.3</u>	26.256 ± 0.000	26.256
Qwen3-14b	0.1458 ± 0.0005	0.1461	0.7226 ± 0.0147	0.7311	4366.8 ± 0.0000	4366.8	26.256 ± 0.000	26.256
Qwen3-8b	<u>0.1468 ± 0.0008</u>	<u>0.1472</u>	0.7268 ± 0.0051	0.7307	4132.2 ± 406.34	4366.8	26.256 ± 0.000	26.256
Qwen2.5-7b	0.1427 ± 0.0021	0.1438	0.7210 ± 0.0130	0.7307	4080.4 ± 65.04	4184.1	26.256 ± 0.000	26.256
<i>Backbone: KIMI-K2</i>								
OpenEvolve	0.1339 ± 0.010	0.1453	0.6980 ± 0.005	0.7012	3966.1 ± 368.9	4237.2	23.443 ± 0.922	24.498
GEPA	0.1287 ± 0.001	0.1297	<u>0.7043 ± 0.003</u>	0.7071	3748.1 ± 126.7	3891.0	23.049 ± 0.932	24.120
AdaEvolve	0.1312 ± 0.005	0.1373	0.6996 ± 0.021	0.7139	3761.1 ± 193.5	3984.0	<u>26.242 ± 0.019</u>	26.256
EvoX	0.1337 ± 0.010	0.1451	0.7038 ± 0.015	0.7181	3613.9 ± 143.6	3759.3	25.436 ± 1.229	<u>26.233</u>
Levi	0.1446 ± 0.000	0.1449	0.6654 ± 0.013	0.6843	<u>4201.6 ± 0.000</u>	4201.6	24.342 ± 0.034	24.498
SPECEVO	↓53% (\$)		↓68% (\$)		↓63% (\$)		↓67% (\$)	
Qwen3-30b	0.1464 ± 0.001	0.1470	0.7034 ± 0.016	<u>0.7201</u>	4143.2 ± 178.8	4255.3	26.212 ± 0.076	26.256
Qwen3-14b	<u>0.1457 ± 0.0006</u>	<u>0.1464</u>	0.7080 ± 0.0237	0.7217	4183.0 ± 175.7	<u>4385.9</u>	26.256 ± 0.000	26.256
Qwen3-8b	0.1438 ± 0.0002	0.1440	0.6909 ± 0.0282	0.7072	4398.9 ± 22.40	4424.8	26.240 ± 0.013	26.256
Qwen2.5-7b	0.1448 ± 0.0008	0.1453	0.6996 ± 0.0234	0.7130	4032.5 ± 354.4	4237.2	26.256 ± 0.000	26.256

H.3 DETAILED RESULTS FOR EACH CO-BENCH PROBLEM CATEGORY

Figure 14 summarizes performance across the eight CO-Bench categories, while Table 14 provides the complete results for all 36 problems. SPECEVO achieves the highest category-level score on five categories: Packing, Cutting, Scheduling, Routing, and Tree problems, with particularly pronounced advantages on Cutting and Tree problems.

The per-problem results further show that these improvements are not concentrated within a single problem family. SPECEVO ranks first on 20/36 problems, including strong results on the assortment,

Table 13: API cost in USD under the fixed three-hour wall-clock budget on systems optimization tasks. Each entry reports the mean and standard deviation over three independent runs. Lower is better.

Strategy	EPLB ($\uparrow$)	LLM-SQL ($\uparrow$)	Transaction ($\uparrow$)	PRISM ($\uparrow$)
<i>Backbone: GPT-5</i>				
OpenEvolve	7.69 $\pm$ 0.36	5.16 $\pm$ 0.95	2.48 $\pm$ 0.25	6.38 $\pm$ 0.25
GEPA	10.04 $\pm$ 1.26	5.59 $\pm$ 0.43	2.38 $\pm$ 0.52	4.03 $\pm$ 0.37
AdaEvolve	8.26 $\pm$ 0.11	6.44 $\pm$ 0.40	2.46 $\pm$ 0.39	6.22 $\pm$ 0.19
EvoX	9.52 $\pm$ 0.57	5.19 $\pm$ 0.23	2.47 $\pm$ 0.31	5.97 $\pm$ 0.22
Levi	2.90 $\pm$ 0.25	1.65 $\pm$ 0.12	1.20 $\pm$ 0.21	2.42 $\pm$ 0.98
SpecEvo	2.87 $\pm$ 0.38	1.62 $\pm$ 0.07	1.21 $\pm$ 0.27	2.41 $\pm$ 1.00
<i>Backbone: KIMI-K2</i>				
OpenEvolve	2.25 $\pm$ 0.32	1.64 $\pm$ 0.05	0.90 $\pm$ 0.15	1.97 $\pm$ 0.27
GEPA	1.88 $\pm$ 0.29	1.55 $\pm$ 0.04	1.02 $\pm$ 0.28	1.69 $\pm$ 0.12
AdaEvolve	2.04 $\pm$ 0.07	1.73 $\pm$ 0.27	1.15 $\pm$ 0.23	1.96 $\pm$ 0.14
EvoX	2.24 $\pm$ 0.05	1.77 $\pm$ 0.13	1.14 $\pm$ 0.31	2.16 $\pm$ 0.10
Levi	1.01 $\pm$ 0.17	0.51 $\pm$ 0.03	0.40 $\pm$ 0.18	0.62 $\pm$ 0.00
SpecEvo	0.99 $\pm$ 0.20	0.53 $\pm$ 0.02	0.39 $\pm$ 0.19	0.65 $\pm$ 0.00

Euclidean Steiner, flow-shop scheduling, and periodic vehicle-routing problems. In contrast, the Assignment category is nearly saturated across all methods, while the results on Facility Location are more mixed. These results demonstrate that SPECEVO generalizes effectively across heterogeneous combinatorial structures rather than specializing in a narrow class of optimization problems.

H.4 DETAILED RESULTS ON LSR-SYNTH

We provide a more detailed analysis of the results on LSR-Synth. In addition to normalized mean squared error (NMSE), we report numeric precision, denoted by $\text{Acc}_{0.1}$ following the evaluation protocol of LLM-SRBench (Shojaee et al., 2025).

For a discovered equation, accuracy to tolerance τ is defined as

$$\text{Acc}_\tau = \mathbb{I} \left[\max_{1 \leq i \leq N_{\text{test}}} \left| \frac{\hat{y}_i - y_i}{y_i} \right| \leq \tau \right], \quad (16)$$

where $\hat{y}_i$ and y_i denote the predicted and ground-truth values, respectively. We use $\tau = 0.1$ and report the percentage of problems satisfying this criterion. Unlike NMSE, which measures the average magnitude of prediction errors, $\text{Acc}_{0.1}$ provides a stricter measure that requires the discovered equation to maintain bounded relative error across the entire test set. We compute this metric separately on the in-distribution (ID) and out-of-distribution (OOD) test sets to assess both interpolation accuracy and extrapolation ability.

Table 15 summarizes the domain-level results under the same fixed budget of 500 candidate evaluations. SpecEvo consistently achieves the lowest NMSE across all domains on both the ID and OOD splits, while also attaining the strongest numeric precision in most settings. The improvements are particularly clear in biology and physics, where discovering accurate equations requires searching over heterogeneous functional structures. The strong OOD performance further suggests that the discovered equations capture generalizable relationships rather than merely fitting the observed samples.

I PROMPT TEMPLATES

This appendix reports the prompt templates used by the three functional roles in SPECEVO.

I.1 SPECULATOR

The Speculator uses the lightweight model for high-throughput exploration: it creates initial variants, reviews archived parents, and applies several mutation and crossover operators. Operator diversity prevents repeated generic rewrites, while the shared output contract ensures that every accepted candidate has both executable code and an embeddable strategy description.

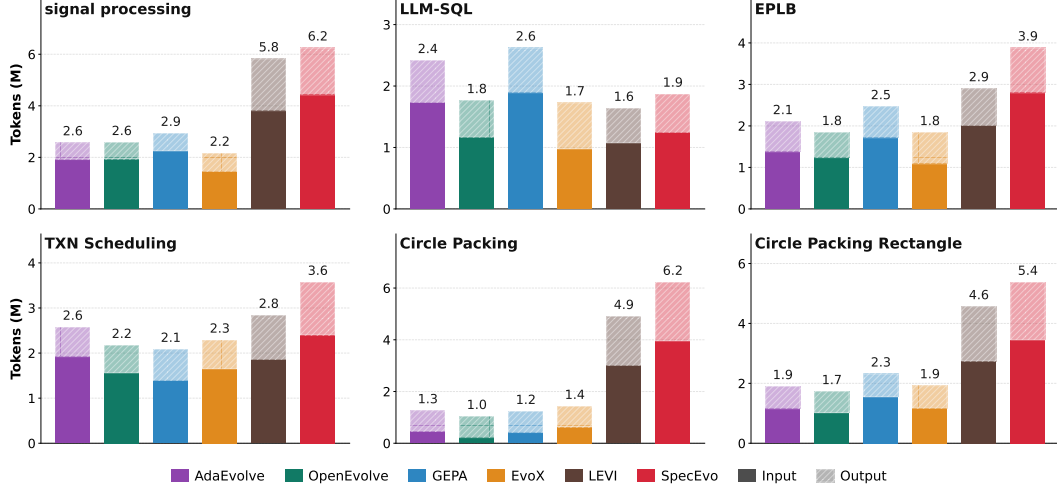


Figure 13: Input and output token usage across six representative tasks using GPT-5 as the frontier-model backbone. Total token consumption is reported in millions (M) for each method.

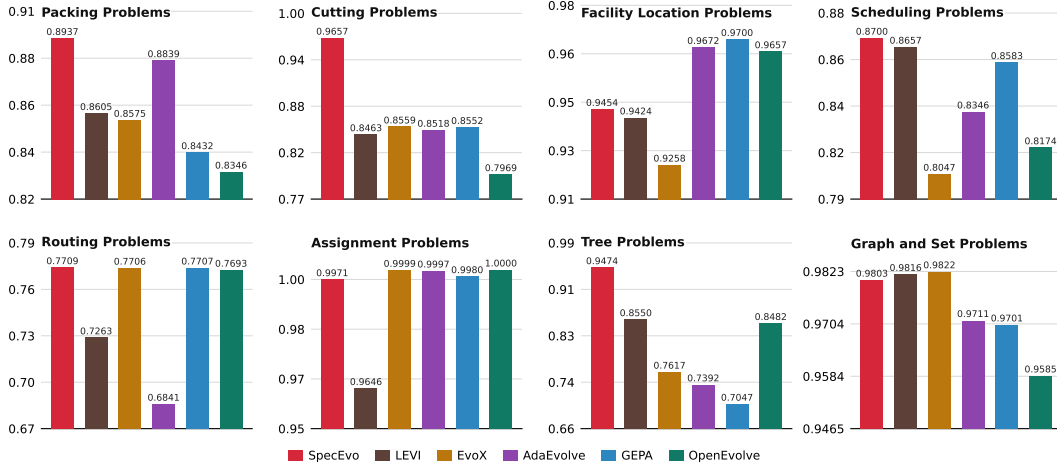


Figure 14: Category-level performance on CO-Bench. Each bar represents the average normalized score over all problems within the corresponding category. Higher is better.

Table 14: Per-task normalized scores on the 36 CO-Bench tasks. Higher scores indicate better performance.

Task	OpenEvolve	GEPA	AdaEvolve	EvoX	Levi	SPECEVO
Packing Problems						
Bin packing – one-dimensional	0.8963	0.8903	<u>0.9213</u>	0.8979	0.8870	0.9863
Multi-Demand Multidimensional Knapsack	0.4473	0.5196	<u>0.6234</u>	0.5893	0.6090	0.6763
Multidimensional knapsack problem	0.9867	0.9846	<u>0.9890</u>	0.9866	<u>0.9890</u>	0.9932
Container loading	0.8362	<u>0.8392</u>	0.8281	0.8156	0.8396	0.8329
Container loading with weight restrictions	0.3540	0.3577	0.6162	<u>0.3777</u>	0.2808	0.2808
Packing unequal circles	0.9666	0.9666	0.9666	1.0000	<u>1.1209</u>	1.3333
Packing unequal circles area	1.0293	1.0913	<u>1.0847</u>	1.0574	1.0686	1.0671
Packing unequal rectangles and squares	<u>0.9616</u>	0.9061	0.9855	<u>0.9616</u>	<u>0.9616</u>	0.8823
Packing unequal rectangles and squares area	1.0338	<u>1.0337</u>	0.9402	1.0315	0.9880	0.9912
Cutting Problems						
Assortment problem	0.4164	<u>0.5650</u>	0.5545	0.5060	0.5562	1.0027
Constrained guillotine cutting	0.8709	0.9247	<u>0.9569</u>	0.9892	0.9176	0.9354
Unconstrained guillotine cutting	0.9460	0.9829	0.9636	<u>0.9777</u>	0.9571	0.9685
Constrained non-guillotine cutting	<u>0.9542</u>	0.9481	0.9323	0.9508	<u>0.9542</u>	0.9562
Facility Location Problems						
Capacitated warehouse location	0.8732	0.9122	0.8763	0.7245	<u>0.9232</u>	0.9485
Uncapacitated warehouse location	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Capacitated p -median	<u>0.9896</u>	0.9677	0.9928	0.9789	0.8475	0.8329
Uncapacitated p -median	<u>0.9998</u>	1.0000	<u>0.9998</u>	1.0000	0.9990	1.0000
Scheduling Problems						
Aircraft landing	0.9026	0.9047	0.9827	<u>0.9144</u>	0.8976	0.8820
Crew scheduling	<u>0.5285</u>	0.4682	0.5294	0.2939	0.5132	0.5132
Common due date scheduling	0.8887	0.9468	0.9271	0.9902	0.9802	<u>0.9867</u>
Flow shop scheduling	0.6364	<u>0.9356</u>	0.6279	0.8585	0.9300	0.9694
Hybrid Reentrant Shop Scheduling	<u>0.9917</u>	<u>0.9917</u>	<u>0.9917</u>	<u>0.9917</u>	<u>0.9917</u>	0.9918
Job shop scheduling	0.8116	0.8238	0.8416	0.7849	<u>0.8501</u>	0.8602
Open shop scheduling	0.9621	0.9370	<u>0.9421</u>	0.7991	0.8971	0.8868
Routing Problems						
Traveling salesman problem	0.9465	<u>0.9641</u>	0.9543	0.9494	0.9691	0.9211
Period vehicle routing problem	0.7527	0.7392	0.4892	<u>0.7538</u>	0.6010	0.7828
Resource-constrained shortest path	0.6087	0.6087	0.6087	0.6087	0.6087	0.6087
Assignment Problems						
Assignment problem	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Generalised assignment problem	1.0000	0.9960	0.9994	<u>0.9998</u>	0.9292	0.9941
Tree Problems						
Euclidean Steiner problem	<u>0.6965</u>	0.4095	0.4784	0.5310	0.7101	0.8947
Corporate structuring	<u>0.9999</u>	<u>0.9999</u>	<u>0.9999</u>	0.9923	<u>0.9999</u>	1.0000
Graph and Set Problems						
Maximal Independent Set	0.9347	0.9202	<u>0.9275</u>	0.9202	0.9261	0.9130
Graph colouring	0.8835	0.9382	<u>0.9500</u>	1.0000	1.0000	1.0000
Equitable partitioning problem	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
Set partitioning	0.9912	1.0000	0.9912	<u>0.9997</u>	0.9898	0.9924
Set covering	0.9831	0.9921	0.9869	0.9911	<u>0.9923</u>	0.9960

Table 15: Performance comparison on LSR-Synth under a fixed budget of 500 candidate evaluations. Performance metrics include numeric precision ($\text{Acc}_{0.1}$) and normalized mean squared error (NMSE).

Domain	Method	NMSE ($\downarrow$)		Acc. _{0.1} (%) ($\uparrow$)	
		ID	OOD	ID	OOD
Chemistry	OpenEvolve	5.27e-06	4.99e-02	33.3	41.7
	GEPA	1.72e-05	2.69e-01	19.4	38.9
	AdaEvolve	2.48e-06	6.05e-02	33.3	41.7
	EvoX	6.87e-06	8.68e-02	27.8	44.4
	Levi	5.80e-06	1.17e-01	27.8	42.5
	SPECEVO	4.57e-07	1.68e-02	41.7	55.6
Biology	OpenEvolve	6.33e-05	7.48e-01	8.3	12.5
	GEPA	8.79e-05	1.92e+00	8.3	12.5
	AdaEvolve	9.92e-06	1.28e-01	12.5	16.7
	EvoX	3.79e-05	5.83e-01	8.3	16.7
	Levi	3.87e-6	1.96e-02	8.3	12.5
	SPECEVO	3.34e-07	3.74e-03	20.8	29.2
Physics	OpenEvolve	3.29e-04	6.93e-03	4.5	11.4
	GEPA	1.53e-03	2.82e-02	0.0	6.8
	AdaEvolve	2.79e-04	1.32e-02	6.8	11.4
	EvoX	6.66e-04	5.32e-02	6.8	6.8
	Levi	1.38e-05	2.23e-03	5.6	12.8
	SPECEVO	2.92e-06	4.75e-04	20.5	20.5
Material Science	OpenEvolve	5.92e-08	2.44e-06	48.0	96.0
	GEPA	1.86e-07	7.57e-05	28.0	96.0
	AdaEvolve	6.32e-07	1.18e-04	44.0	72.0
	EvoX	9.05e-08	1.63e-05	36.0	96.0
	Levi	5.73e-07	8.36e-06	36.0	68.0
	SPECEVO	4.84e-08	1.68e-06	52.0	88.0

Speculator — Prompt for population-variant initialization

You are an expert algorithm designer helping to construct a diverse initial population for an optimization problem.

{Task Description}

{Function Signature}

{Inspiration Programs: code and score for one or two seeds}

Write an improved, complete, and self-contained implementation of the required function. Use the inspiration programs to identify useful mechanisms, but produce a meaningfully different seed rather than copying one. Aim for a variant that is both feasible and competitive; check the syntax carefully before returning it.

{Output Format}

Return exactly two sections in this order: ## Description and ## Code. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced `python` block with the complete, runnable, self-contained program. Match the required function signature exactly and include every necessary import.

Speculator — Prompt for parent analysis

You are a code reviewer identifying the highest-impact bottlenecks in a candidate optimization program.

{Task Description}

{Function Signature}

{Parent Description, Parent Score, and Parent Program}

Review why the program likely does not score higher. Write exactly three sections and no code:

1. **Algorithm summary:** in 1–2 sentences, name the algorithmic class, its main construction or search strategy, and its key data structures.
2. **Top three bottlenecks, ranked by expected impact:** for each, identify the exact routine, loop, variable, update rule, repair step, acceptance rule, data structure, or constant; explain how it limits score, feasibility, exploration, exploitation, runtime, or objective alignment; and label its expected impact High, Medium, or Low.
3. **Suggested changes:** give exactly three directly implementable changes, one for each bottleneck and in the same order.

Prefer algorithmic bottlenecks over cosmetic issues or isolated constant tuning. Keep the review under 250 words and avoid vague recommendations.

Speculator — Prompt for general mutation

You are an expert in optimization heuristics improving a candidate algorithm.

{Task Description}

{Function Signature}

{Parent Program and Parent Score}

{Archive Inspirations: descriptions and scores only}

{Optional Lessons Learnt from the Advisor}

Preserve the parent’s useful structure, but make one concrete algorithmic change that is likely to improve the score. In ## Analysis, give exactly four parts: **Components** (the 3–5 main phases), **Strengths** (what should be preserved), **Weaknesses** (the most likely bottleneck, citing a concrete code element), and **Plan** (one sentence stating the mutation and why it should help). The change must be a real behavioral improvement, not renaming, reformatting, comment editing, random-seed replacement, or isolated constant retuning. Treat inspirations as high-level ideas and do not assume access to their hidden code.

{Output Format}

Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The analysis must follow the operator-specific structure below. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced `python` block with the complete, runnable, self-contained program. The required function signature must match exactly; include every import and do not use placeholders, ellipses, or incomplete branches.

Speculator — Prompt for focused single-weakness mutation

You are an expert in optimization heuristics making a surgical improvement to an already functional candidate.

{Task Description}

{Function Signature}

{Parent Program and Parent Score}

{Archive Inspirations: descriptions and scores only}

{Optional Lessons Learnt from the Advisor}

Choose exactly one weakness and fix it while preserving the parent’s algorithmic identity and every unrelated component. In ## Analysis, give exactly three parts: **Tightest constraint** (cite the specific limiting code element), **Fix** (one sentence describing the minimum meaningful change), and **Preserved** (the top-level components that remain unchanged). Do not add unrelated improvements, retune unrelated constants, alter only the seed, or introduce a helper that is not required by the selected fix.

{Output Format}

Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The analysis must follow the operator-specific structure below. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced `python` block with the

complete, runnable, self-contained program. The required function signature must match exactly; include every import and do not use placeholders, ellipses, or incomplete branches.

Speculator — Prompt for mechanism-swap mutation

You are an expert in optimization heuristics replacing one interchangeable mechanism inside a candidate algorithm.

{Task Description}

{Function Signature}

{Parent Program and Parent Score}

{Archive Inspirations: descriptions and scores only}

{Optional Lessons Learnt from the Advisor}

Identify one local slot whose mechanism can be replaced without changing the overall algorithm. Examples include uniform with stratified initialization, a fixed step with an adaptive step or line search, greedy with Metropolis acceptance, a fixed with a stagnation-triggered restart, or a simple with a slack-aware repair rule. In ## Analysis, give exactly three parts: **Mechanism identified**, **Old versus new** (“Currently uses X; replace it with Y because Z”), and **Risk** (the main failure mode and its guard). Change only the selected mechanism and preserve the signature, I/O contract, and parent structure.

{Output Format}

Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The analysis must follow the operator-specific structure below. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced `python` block with the complete, runnable, self-contained program. The required function signature must match exactly; include every import and do not use placeholders, ellipses, or incomplete branches.

Speculator — Prompt for analysis-targeted mutation

You are an expert in optimization heuristics implementing a previously identified improvement.

{Task Description}

{Function Signature}

{Parent Program and Parent Score}

{Cached Analysis of the Parent}

{Archive Inspirations: descriptions and scores only}

{Optional Lessons Learnt from the Advisor}

Read the cached analysis and choose exactly one suggested change: the one most likely to improve the score. Do not invent another direction. In ## Analysis, give exactly three parts: **Chosen bottleneck** (quote or precisely restate it), **Implementation plan** (one sentence describing the structural edit), and **What stays unchanged**. Implement exactly that one structural change and keep the rest of the working parent stable.

{Output Format}

Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The analysis must follow the operator-specific structure below. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced `python` block with the complete, runnable, self-contained program. The required function signature must match exactly; include every import and do not use placeholders, ellipses, or incomplete branches.

Speculator — Prompt for structural crossover

You are an expert in optimization heuristics recombining two candidate algorithms.

{Task Description}

{Function Signature}

{Parent A: program and score}

{Parent B: program and score}

```
{Archive Inspirations:  descriptions and scores only}
```

```
{Optional Lessons Learnt from the Advisor}
```

Produce a coherent structural hybrid by selecting the strongest compatible mechanism from each parent and reintegrating them into a single data flow, objective, and constraint-handling logic. In ## Analysis, give exactly three parts: **Component table** (initialization, optimization core, and constraint handling, each attributed to A, B, or a hybrid), **Compatibility note**, and **Expected improvement**. Do not merely paste blocks together; define every helper and preserve the required I/O contract.

```
{Output Format}
```

Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The analysis must follow the operator-specific structure below. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced python block with the complete, runnable, self-contained program. The required function signature must match exactly; include every import and do not use placeholders, ellipses, or incomplete branches.

Speculator — Prompt for component-swap crossover

You are an expert in optimization heuristics transplanting one component from a donor into a base algorithm.

```
{Task Description}
```

```
{Function Signature}
```

```
{Base Parent:  program and score}
```

```
{Donor Parent:  program and score}
```

```
{Archive Inspirations:  descriptions and scores only}
```

```
{Optional Lessons Learnt from the Advisor}
```

Keep the base parent as the skeleton: preserve its overall control flow, data flow, termination logic, and I/O contract. Transplant exactly one donor routine, block, update rule, repair step, schedule, or data structure that is clearly stronger than the base equivalent, with only the glue changes required for integration. In ## Analysis, give exactly three parts: **Donor component chosen**, **Glue work**, and **What is not changed**. Keep the base recognizable and make no extra improvements.

```
{Output Format}
```

Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The analysis must follow the operator-specific structure below. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced python block with the complete, runnable, self-contained program. The required function signature must match exactly; include every import and do not use placeholders, ellipses, or incomplete branches.

Speculator — Prompt for Navigator fan-out variants

You are an expert in optimization heuristics exploring the neighborhood of an accepted Navigator proposal.

```
{Task Description}
```

```
{Function Signature}
```

```
{Base Navigator Program and Score}
```

Generate one variant that keeps the base program’s core algorithmic approach intact while making targeted modifications to constants and thresholds, secondary heuristics, edge-case handling, or implementation details. Explore a nearby region of the solution space without discarding the novel strategy introduced by the Navigator. Handle boundary cases, include all imports, and check the syntax carefully.

```
{Output Format}
```

Return exactly two sections in this order: ## Description and ## Code. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the algorithmic paradigm, its key data structures, and its distinguishing mechanism. The code section must contain one fenced python block with the complete, runnable, self-contained program. Match the required function signature exactly and include every necessary import.

Speculator — Prompt for missing-description recovery

You are summarizing a generated program whose code was parsed successfully but whose strategy description was missing or too short.

{Generated Program}

Write one flowing paragraph of 2–4 sentences and fewer than 80 words. Name the specific algorithmic paradigm, the concrete key data structures, and the distinguishing trick or heuristic that separates the program from a textbook version of that paradigm. Do not restate the problem, inputs, outputs, or function signature; do not use headings, bullet points, tag-style openers, or boilerplate such as “This solution” or “The function.” Return the paragraph only.

I.2 NAVIGATOR

The Navigator uses the frontier model for the small number of decisions that require population-level reasoning. It first creates distinct seed families and later selects Synthesis, Surgical, or Reframe according to stagnation rate.

Navigator — Prompt for population initialization

You are an expert in the domain of optimization heuristics helping to design algorithms that can effectively solve optimization problems.

{Task Description}

{Function Signature}

{Previously Accepted Seeds: complete programs and scores}

Design a solution using a fundamentally different algorithm from every existing seed. First analyze which paradigm each seed uses, what information it exploits, and what relevant information it ignores. Then design from first principles using a different modeling, decomposition, construction, or search strategy. Do not make a minor variation, tune parameters, reorder existing logic, or refactor the same core algorithm.

{Output Format}

Return ## Description followed by ## Code. The description is a 2–4 sentence paragraph (at most 80 words) stating the paradigm, key data structures, and distinguishing mechanism. The code must be complete, runnable, self-contained, and expose the exact required signature.

Navigator — Prompt for Synthesis

You are an expert research lead synthesizing complementary mechanisms from competitive solutions.

{Task Description}

{Function Signature}

{Archive State: evaluation count, occupied niches, and stagnation}

{Top Three Anchors: description, score, and complete program}

{Five Additional Inspirations: descriptions and scores only}

The search is only mildly stalled. Synthesize, rather than invent, one program that coherently combines 2–3 concrete mechanisms from different anchors and plausibly beats each anchor individually. After the line `MOVE: SYNTHESIS`, include exactly three analysis parts: **Component table** (initialization, optimization core, and constraint repair, with anchor provenance), **Coherence note** (shared data layout, units, and call order), and **Why this should beat all anchors** (one sentence per anchor).

{Output Format}

Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The first line of the analysis must be `MOVE: SYNTHESIS`. The description must be one plain-prose paragraph of 2–4 sentences (at most 80 words) naming the paradigm, key data structures, and distinguishing mechanism. The code must be complete and runnable, reproduce the required signature exactly, define every helper, and include every import.

Navigator — Prompt for Surgical improvement

You are an expert research lead applying one precise structural improvement to the current champion.

{Task Description}

```
{Function Signature}
{Archive State:  evaluation count, occupied niches, and stagnation}
{Current Champion:  description, score, and complete program}
{Five Additional Inspirations:  descriptions and scores only}
The search is moderately stalled, so improve the champion without changing its algorithmic family. After
the line MOVE:  SURGICAL, include exactly four analysis parts: Tightest constraint (cite the exact
limiting mechanism), Structural fix (one local behavioral change), Preservation list, and Expected delta
(benefit, main risk, and guard). Implement exactly one local structural change; prefer using existing score,
feasibility, slack, loss, or quality signals over a blind heuristic, and do not add unrelated improvements.
{Output Format}
Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The
first line of the analysis must be MOVE:  SURGICAL. The description must be one plain-prose paragraph
of 2–4 sentences (at most 80 words) naming the paradigm, key data structures, and distinguishing
mechanism. The code must be complete and runnable, reproduce the required signature exactly, define
every helper, and include every import.
```

Navigator — Prompt for Reframe

You are an expert research lead proposing a new algorithmic family when the current population has plateaued.

```
{Task Description}
{Function Signature}
{Archive State:  evaluation count, occupied niches, and stagnation}
{Top Two Anchors:  description, score, and complete program}
{Five Additional Inspirations:  descriptions and scores only}
The search is deeply stalled. Design a fundamentally different paradigm whose data structures, control
flow, and termination rule do not appear in the anchors or inspirations. After the runtime line MOVE:
REFRAME, include exactly four analysis parts: Paradigm name (use the textbook name), Why it fits the
problem (two problem-specific sentences), Why the current anchors miss it (the shared assumption
being dropped), and Risk (the main implementation pitfall and guard). Forbidden moves are constant
retuning, subroutine grafting, and variable renaming.
{Output Format}
Return exactly three sections in this order: ## Analysis, ## Description, and ## Code. The
first line of the analysis must be MOVE:  REFRAME. The description must be one plain-prose paragraph of
2–4 sentences (at most 80 words) naming the paradigm, key data structures, and distinguishing
mechanism. The code must be complete and runnable, reproduce the required signature exactly, define
every helper, and include every import.
```

I.3 ADVISOR

The Advisor converts deterministic, niche-level search statistics and accumulated failures into short natural-language guidance.

Advisor — Prompt for population-level reflection

You are a lessons-learned advisor reviewing the search trajectory of an evolutionary code-optimization system. Write a short prescriptive note that a mutation model will read verbatim before some future attempts. Do not restate the task or generic constraints already enforced by the evaluator.

```
{Task Description}
{Function Signature}
{Current State:  best score, evaluations, accept rate, and stagnation}
{Improving Niches:  frontiers advanced in the current window}
{Saturated Niches:  recent attempts, unchanged frontiers, and scores}
{Accumulated Failure Knowledge:  recurrence count and example}
```

{Previous Advice}

Write exactly the following four sections, in this order, using the literal headers and no other Markdown:

WORKING: In 1–2 sentences, name the concrete algorithmic structure or mechanism shared by improving niches and leaders. Describe the algorithm, not the search process; if nothing is clearly working, say so.

SATURATED: In 1–2 sentences, name the busy strategy families whose frontiers no longer improve and should be de-emphasized. If there is no clear saturation, write “none.”

AVOID: Give 8–10 anti-patterns supported by the most recurrent failures. Skip generic defensive advice unless that failure recurs heavily.

Keep the complete answer under 300 words. Write each section as one short paragraph after its header, with no preamble, extra headers, or bullet characters. Return advice only; do not output code.

J QUALITATIVE ANALYSIS

This appendix examines the programs that SPECEVO produces and the failures that arise on two representative tasks: Circle Packing Rectangle, a continuous geometric optimization problem, and EPLB, a discrete resource allocation problem for mixture of experts inference. For each task, we examine the complete evaluation log from one search run. The log records every candidate program, the role and model that produced it, its execution status, and the message returned by the evaluator. We organize the analysis along three axes: the errors that caused candidates to be rejected, the families of algorithmic ideas explored by the population, and the guidance distilled by the Advisor from the accumulated failures.

The analysis is deliberately qualitative. We characterize failure classes by what goes wrong rather than reporting exact frequencies, and order them from most to least common within each task. We describe idea families by the role they play in the resulting algorithm. Because a program usually combines several idea families, these families are complementary rather than mutually exclusive. Throughout, we use four coarse prevalence labels, *ubiquitous*, *common*, *occasional*, and *rare*, to indicate how widely each mechanism appears across the population.

J.1 CIRCLE PACKING RECTANGLE

The task asks for the placement of 21 nonoverlapping circles inside a rectangle of fixed perimeter while maximizing the sum of their radii. It combines continuous geometry with a strict feasibility contract: even a configuration that violates feasibility by only one part in a thousand receives no score.

Recurring errors. Table 16 groups the rejected candidates. By far the most common failure is a violation of the geometric contract, almost always caused by a pair of intersecting circles. These failures are not near misses caused by a tolerance that is one digit too strict. Instead, they arise when a program inflates the radii or rescales a completed layout to fill the container, then returns the result without rechecking pairwise feasibility. Consequently, the achieved separation misses the required threshold by a visible margin rather than by an epsilon. Adjusting the tolerance would therefore rescue almost none of these candidates; the missing safeguard is the check itself.

The second most common failure concerns the output contract. The evaluator expects an array with 21 rows, each containing a centre and a radius, but programs lose this shape inside helper routines, early return branches, and especially fallback paths. These paths are often written last and exercised only when the main path fails, so the author may never test them. The remaining classes are ordinary programming faults, with one recurring curiosity: candidates sometimes emit typographic Unicode symbols, such as a minus sign or a less than or equal sign copied from prose. These symbols make the program unparseable before any geometry is evaluated.

Two properties of these failures motivate the persistent guidance design. First, they reflect the task and the model rather than any particular search architecture. We ran the same task under five alternative coordination policies in our relay study. Overlapping circles were the leading failure under every policy without exception, and the same secondary classes repeatedly appeared. Second, these failures did not become less frequent as the runs progressed. When each run was divided into quarters, the rejection counts remained essentially flat. The population improved in score while the same failures continued to recur because nothing in the loop carried forward evidence about what had

Table 16: Failure classes among rejected candidates on Circle Packing Rectangle, ordered from most to least common. Each rejected candidate belongs to exactly one class.

Class	What goes wrong
Feasibility	Two circles intersect after the radii are inflated or the layout is rescaled, but the program returns without rechecking pairwise feasibility
Output shape	The returned array does not contain 21 rows with three values each, usually because a helper or fallback path uses a different representation
Type	Lists and arrays are combined in arithmetic, or a coordinate is missing or infinite
Name	A variable or import is used before being defined, often within a nested helper or early return branch
Runtime	The program exhausts the evaluation time limit instead of stopping after convergence
Numeric	An array is used directly in a truth test, or a reduction is applied to an empty array
Indexing	An index exceeds the array bounds while circle pairs are enumerated
Syntax	Typographic Unicode symbols or malformed code prevent parsing

already failed. In the SPEC-EVO run, by contrast, the rejection rate dropped substantially from the unguided initialization phase to the subsequent evolutionary phase, once an Advisor note became available for inclusion in Speculator prompts.

The relay logs also distinguish the two model tiers, revealing a qualitative difference rather than a gap in overall competence. When both tiers receive comparable numbers of calls, their rejection rates are close and their overlap rates are nearly indistinguishable. The remaining failures, however, differ. Timeouts occur almost entirely in the strong model, whose programs are longer and rely on expensive global optimizers, whereas indexing and shape errors are concentrated in the lightweight model. The strong model tends to produce code that is correct but slow, while the lightweight model tends to produce code that is fast but brittle. The Speculate Then Consolidate workflow is designed to exploit this distinction: cheap parallel exploration supplies volume, while sparse expensive interventions supply structure.

Families of code ideas. Table 17 lists the mechanisms explored by the population, grouped by their role. *Construction* mechanisms determine the initial circle positions. Nearly every candidate begins with a structured layout, typically hexagonal rows with alternating offsets and an explicit row pattern, rather than random points. Most candidates then search over the aspect ratio of the container because the objective couples the rectangle shape with the number of circles that fit in each row. *Refinement* mechanisms improve the configuration through descent on a smooth penalty for pairwise intersections, simulated annealing with a cooling schedule and stochastic acceptance, or iterative separation in which every overlapping pair produces damped opposing displacements. *Repair* mechanisms restore feasibility by rescaling the entire configuration to satisfy the perimeter constraint, greedily increasing each radius to its nearest neighbor limit, and applying a final shrink pass using the smallest factor that removes all remaining intersections. *Engineering* mechanisms help the program complete evaluation reliably. These include a wall clock guard that returns the best feasible configuration found, priority queues that process the largest radius or violation first, and restart portfolios that retain the best of several independent attempts.

The division of labor between the roles is visible in how these mechanisms are combined. The initialization phase and the lightweight Speculators supply the individual mechanisms and refine them one at a time, so the record improves in small steps through targeted mutation and crossover. Navigator interventions set new records in this run through assembly rather than invention. The final best program initializes a hexagonal layout, relaxes it with damped pairwise separation, greedily increases the radii toward their nearest neighbor limits, adds a backtracking enlargement pass for each circle, and renormalizes the layout to the perimeter constraint after every stage. A final shrink pass guarantees feasibility before the program returns. Every component already existed somewhere in the population, yet no candidate using only one mechanism matched the score of their combination.

Advisor feedback. Throughout the run, the Advisor consolidated the accumulated rejections into a short list of prescriptions, ordered by the amount of failure evidence each one addresses. Every

Table 17: Algorithmic mechanisms observed on Circle Packing Rectangle, grouped by their role. A program typically combines mechanisms from every group.

Role	Mechanism	Prevalence and remarks
Construction	Structured lattice or patterned row initialization	Ubiquitous; hexagonal rows with an explicit pattern rather than random points
	Search over the container aspect ratio	Common; the objective couples container shape with row occupancy
Refinement	Simulated annealing with stochastic acceptance	Common; uses a cooling schedule over perturbed configurations
	Continuous penalty descent	Common; penalizes intersections smoothly rather than forbidding them
	Pairwise separation relaxation	Common; applies damped opposing displacements to overlapping pairs
	Enlargement with backtracking for each circle	Occasional; halves the step until an individual radius fits
Repair	Global rescaling to satisfy the perimeter constraint	Common; applied after every stage rather than only at the end
	Final feasibility shrink pass	Common; uses the smallest global factor that removes all intersections
	Greedy radius inflation to the nearest neighbor limit	Occasional; grows each radius to the largest value permitted by its neighbors
	Neighbor queries using a spatial index	Rare; replaces pairwise distance calculations with a spatial tree or triangulation
Engineering	Wall clock guard with a best feasible fallback	Ubiquitous; the only defense against the runtime failure class
	Priority queue processing	Occasional; handles the largest radius or violation first
	Multiple restart portfolio	Occasional; retains the best of several independent attempts

item targets a failure class rather than an individual program. At each checkpoint, the note is rewritten instead of extended, keeping it short enough to be included verbatim in a Speculator prompt.

Consolidated Advisor guidance: Circle Packing Rectangle

1. Ensure that no two circles overlap by keeping the distance between their centers at least equal to the sum of their radii minus the tolerance, as the validator requires.
2. Return exactly 21 circles as an array with 21 rows and three values per row, and preserve this shape through every operation.
3. Convert all coordinates and radii to double precision before arithmetic, and ensure that no value is missing or infinite.
4. Define and import every name before use, and call only documented public interfaces.
5. Keep the total runtime below the evaluation limit by bounding iteration counts and stopping early after convergence.
6. Confirm that an array is nonempty before using it in a truth test or a reduction.
7. Clip indices to the actual array size, especially when enumerating circle pairs.
8. Use only plain ASCII characters, replacing typographic minus and less than or equal symbols with their ASCII equivalents.
9. Clip radii to a small positive lower bound before any geometric computation to avoid division by zero and invalid square roots.

Two features of this guidance are characteristic. First, it is prescriptive rather than diagnostic: it tells subsequent programs what to do in terms that a generation model can act on directly, instead of merely reporting what an earlier program did wrong. Second, it is ordered by the evidence, so the highest ranked advice would have rescued the most candidates. The leading items describe contracts rather than algorithms, and no degree of algorithmic sophistication can substitute for them. Because

Table 18: Failure classes among rejected candidates on EPLB, ordered from most to least common. Each rejected candidate belongs to exactly one class.

Class	What goes wrong
Indexing	A physical or logical index derived from capacity arithmetic exceeds a tensor bound
Name	A module or attribute is used without being imported or defined, most often the priority queue module
Type	A tensor has the wrong element type, or an integer computation exceeds its range
Feasibility	Replica counts do not sum to the budget for every layer, or the placement exhausts the available capacity
Runtime	The placement loop exhausts the evaluation time limit
Shape	A tensor dimension does not match the extent expected by the evaluator
Numeric	A nonscalar tensor is used directly in a truth test
Output shape	The function does not return the three required tensors in the required order
Syntax	A typographic Unicode symbol makes the program unparseable

the note is included in only a fraction of Speculator prompts, the remaining proposals continue to explore without it. This prevents the guidance from collapsing the population into a single defensive coding style.

J.2 EPLB

The task asks programs to place logical experts onto physical GPU slots for mixture of experts inference. Given load statistics for each layer, a program must decide how many replicas each logical expert receives and which slot each replica occupies, subject to a fixed replica budget, a fixed hierarchy of groups, nodes, and GPUs, and a capacity limit for each GPU. Unlike the geometric task, which is continuous with a strict feasibility boundary, EPLB is discrete and requires careful bookkeeping. Three nested notions of position, namely the logical expert, replica, and physical slot, must remain mutually consistent as replica counts are adjusted.

Recurring errors. Table 18 groups the rejected candidates. Its failure profile differs from that of the geometric task in an instructive way. In Circle Packing Rectangle, the dominant failure was a constraint violation within an otherwise valid program. In EPLB, indexing dominates: a physical slot or replica index derived from capacity arithmetic exceeds the bound of the tensor it addresses. The allocation policy is usually sound, but the surrounding bookkeeping is not.

Two of these classes have a single identifiable cause, illustrating how a productive idea can propagate a systematic defect. Name errors overwhelmingly result from one oversight: candidates use a priority queue to schedule every replica on the currently least loaded GPU, implement the queue operations correctly, and omit the import. None of the affected programs includes the import anywhere. Some go further and invent private heap routines in the tensor library that do not exist, illustrating what an undocumented interface looks like when a model needs one but cannot find it.

Type errors are similarly concentrated and trace back to the deterministic hash initialization described below. In this pattern, a large hexadecimal constant is combined with a slot index so that replicas of the same expert are dispersed across devices without a random number generator. The idea is reasonable, and the resulting dispersion is genuinely useful. However, the product silently exceeds the integer range unless it is reduced modulo the number of GPUs before being used as an index. The resulting index is then not merely one position beyond the boundary but astronomically out of bounds. Nearly every overflow rejection in the run came from a program containing such a constant. This example shows why the failure and the idea must be considered together: one line of consolidated guidance can retain the mechanism while removing its defect.

The rejection rate falls steadily throughout the run, with a particularly large drop from the unguided initialization phase to the evolutionary phase that follows. The mechanical faults that dominate the early population, including the missing import, overflowing constant, and invalid slot index, are precisely the recurring signatures accumulated by the Advisor. These faults are largely absent from the later population, even as the same algorithmic ideas continue to be explored.

Table 19: Algorithmic mechanisms observed on EPLB, grouped by the design decision they resolve. A program typically combines mechanisms from every group.

Decision	Mechanism	Prevalence and remarks
Structure	Independent solution for each layer	Ubiquitous; imposed by the interface
	Explicit treatment of the group, node, and GPU hierarchy	Ubiquitous; implements placement as a nested traversal of the hierarchy
Replicas	Allocation by marginal gain or apportionment	Ubiquitous; assigns the next replica where it most reduces the load per replica
	Water filling threshold found by bisection	Common; gives each expert enough replicas to fall below a searched load threshold
	Huntington–Hill apportionment by name	Occasional; implements the classical rule explicitly
Placement	Modular or rotated round robin assignment	Common; disperses the replicas of each expert across devices
	Deterministic hash initialization of the starting slot	Common; provides dispersion without a random number generator but causes the overflow failures
	Priority queue scheduling by load	Occasional; places each replica on the least loaded GPU
	Local swap or exchange refinement	Rare; exchanges two placed replicas after the initial assignment
Feasibility	Capacity clamping with a repair pass	Ubiquitous; fills remaining slots after the main assignment
	Explicit assertion on the replica budget	Common; checks the contract rather than assuming it
	Vectorized tensor operations instead of Python loops	Common; the main defense against the runtime failure class

Families of code ideas. Table 19 lists the mechanisms explored by the population. Every candidate solves each layer independently and respects the group, node, and GPU hierarchy because both properties are imposed by the interface. Within this structure, three substantive design decisions recur.

The first decision is how many replicas each expert receives. Most candidates allocate replicas by marginal gain, repeatedly assigning the next replica to the expert for which it most reduces the load per replica. This is the classical apportionment rule, and some candidates implement it explicitly under its formal name. A substantial minority instead use binary search to find a load threshold, then assign each expert enough replicas to fall below it. This water filling construction reaches the same budget from the opposite direction. The strongest programs combine both approaches, using threshold search to approach the budget and the marginal gain rule to correct the residual. The second decision concerns slot assignment. Common approaches include a modular or rotated round robin walk over the GPUs, which spreads the replicas of each expert across devices, and a priority queue ordered by load, which always places the next replica on the least loaded GPU. The third decision is how to enforce feasibility. Capacity clamping with an explicit repair pass for remaining slots, together with assertions that replica counts still sum to the budget, appears in most candidates. The Advisor therefore reinforces these mechanisms rather than discouraging them.

The division of roles follows the same pattern as in the geometric task, with one instructive difference. The Navigator’s synthesis and surgical interventions consistently produce strong, valid programs and reinforce the incumbent design. Its later reframing interventions, which deliberately introduce a strategy family absent from the current population, score below the incumbent. This is the intended behavior rather than a failure. Reframing is triggered by stagnation to increase diversity, and the archive retains these programs as parents even when they do not immediately improve the record. The final record was set by a lightweight structural crossover that combined replica allocation based on threshold search with the marginal gain correction step and placement that respects hierarchical capacity constraints. An expensive proposal supplied the structure, and many cheap descendants searched around it.

Advisor feedback. The Advisor consolidated the accumulated rejections into the guidance below. The list is more mechanical than the one for the geometric task, reflecting the different failure profile. On EPLB, most rejected candidates use sound allocation policies but fail because of indexing, import, or element type defects. The most effective guidance therefore emphasizes disciplined representations rather than algorithm design.

Consolidated Advisor guidance: EPLB

1. Clip every index to the valid range of the dimension it addresses, keeping physical and logical indices within bounds and consistent with the expected shapes.
2. Import all required modules, including the priority queue module, at the top of the program, and call only documented public interfaces rather than private tensor library routines.
3. Convert tensor values to the intended element type before use, and maintain that type consistently across operations.
4. Ensure that replica counts sum to the replica budget for every layer, and clamp them to both the budget and the available GPU capacity.
5. Bound the number of placement iterations so that the program remains within the evaluation time limit.
6. Give the mapping from logical to physical indices the required three dimensions, with the final extent equal to the largest replica count.
7. Pass tensor shapes as tuples of integers rather than as a single integer.
8. Reduce a tensor to a scalar before using it in a truth test.
9. Return exactly the three required tensors in the order specified by the function signature.
10. Use only plain ASCII characters because a typographic hyphen is not valid syntax.
11. Use floor rounding rather than ceiling rounding in integer division so that the result matches the expected element type.

Comparing the two tasks makes the value of consolidation concrete. In each task, a small number of instructions covers most of the failure evidence, and the leading instruction describes a contract rather than an algorithm. A stronger backbone alone would not be expected to fix these problems: the relay comparison shows that the strong and lightweight models fail at similar overall rates and differ mainly in which contracts they violate. The decisive change is that the evidence is carried forward. At every checkpoint, the guidance is rewritten from the accumulated failure signatures rather than generated anew from the most recent trial. Consequently, a defect observed many evaluations earlier can still shape the next proposal.